

Embedded Systems and IoT

Ingegneria e Scienze Informatiche - UNIBO

a.a 2025/2026

Lecturer: Prof. Alessandro Ricci

[module-3.1]

EMBEDDED SYSTEMS BASED ON SOC AND RTOS

MODULE OUTLINE

- Introduction to embedded systems based on SoCs (System-On-a-Chip)
- Introduction to Real-Time OS
 - Operating Systems — some recall
 - RTOS

FROM MICROCONTROLLERS TO SoC

- So far we focused on embedded systems based on microcontrollers
 - no operating systems
- Now we move from microcontrollers to **system-on-a-chip systems**
 - chips for embedded systems featuring a CPU architecture and enough memory to host an OS, typically a Real-Time OS

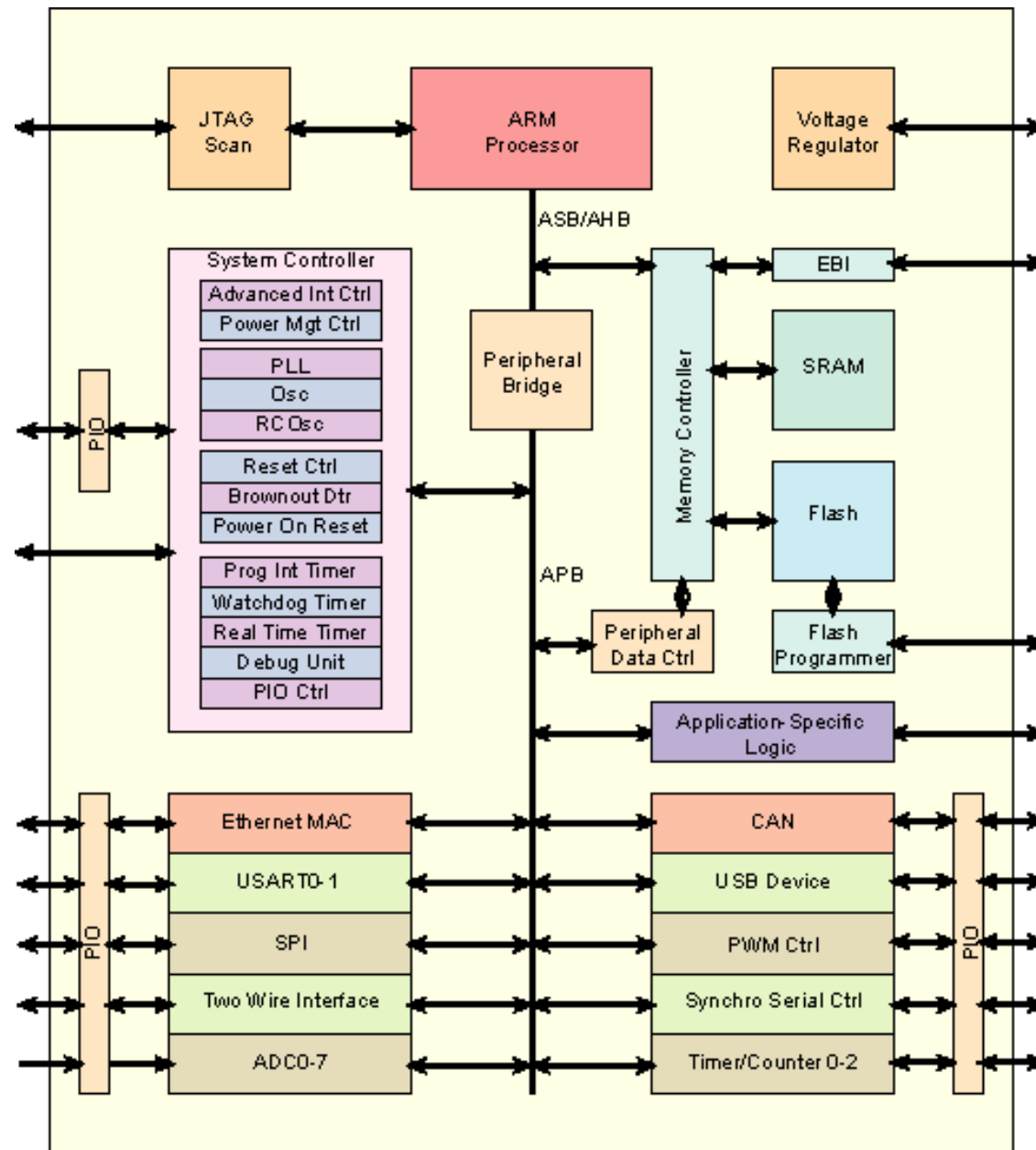
SoC AND SINGLE-BOARD CPU

- **SOC = System-on-a-Chip**
 - an integrated circuit (a chip) that integrates all or most components of a computer or other electronic system
 - CPU, memory, I/O controllers, network interfaces, etc
- Examples:
 - BROADCOM BCM2837 64bit ARMv8 Cortex A53 Quad Core (1.2Ghz)
 - used in Raspberry Pi 3
 - ARM Sitara AM335x SoC - including ARM Cortex-A8 processor
 - used in BeagleBone, Arduino Tre
 - Texas Instruments OMAP3530
 - ...

SoC AS AN INTEGRATED SYSTEM

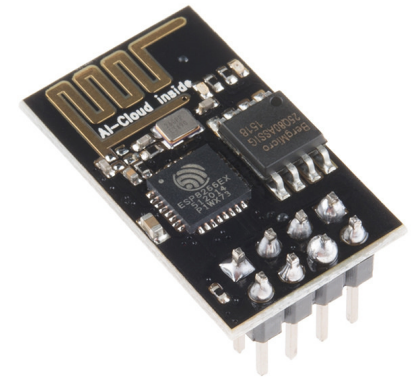
- A SoC integrates:
 - one or multiple processors/cores
 - including microcontrollers, microprocessors, DSP
 - a memory module containing one or multiple blocks of different types:ROM, RAM, EEPROM, FLASH
 - a clock, timers
 - connectors for standard interfaces, such as USB, Ethernet, USART, I2C, SPI
 - radio/network interfaces
 - DAC and ADC
 - ...

SoC



An IOT TINY SOC: ESP 8266

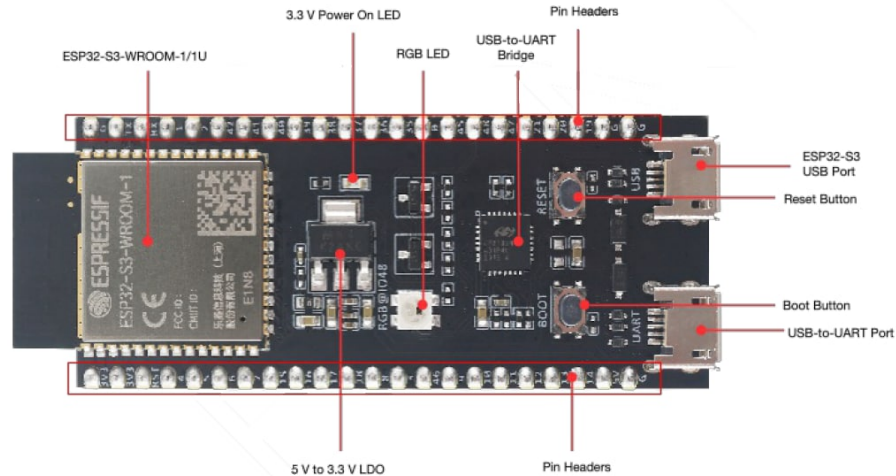
- SoC designed for IoT
- Hardware
 - Processor: L106 32-bit RISC microprocessor core 80 MHz*
 - 64 KiB of instruction RAM, 96 KiB of data RAM
 - 16 GPIO pins
 - SPI, I²C, I²S + UART
 - 10-bit ADC
 - IEEE 802.11 b/g/n Wi-Fi
- Various SDKs and firmwares
 - GCC toolchain, Wiring / Arduino, NodeMCU Lua-based, MicroPython Python based, ESP-Open-RTOS Open source FreeRTOS-based...



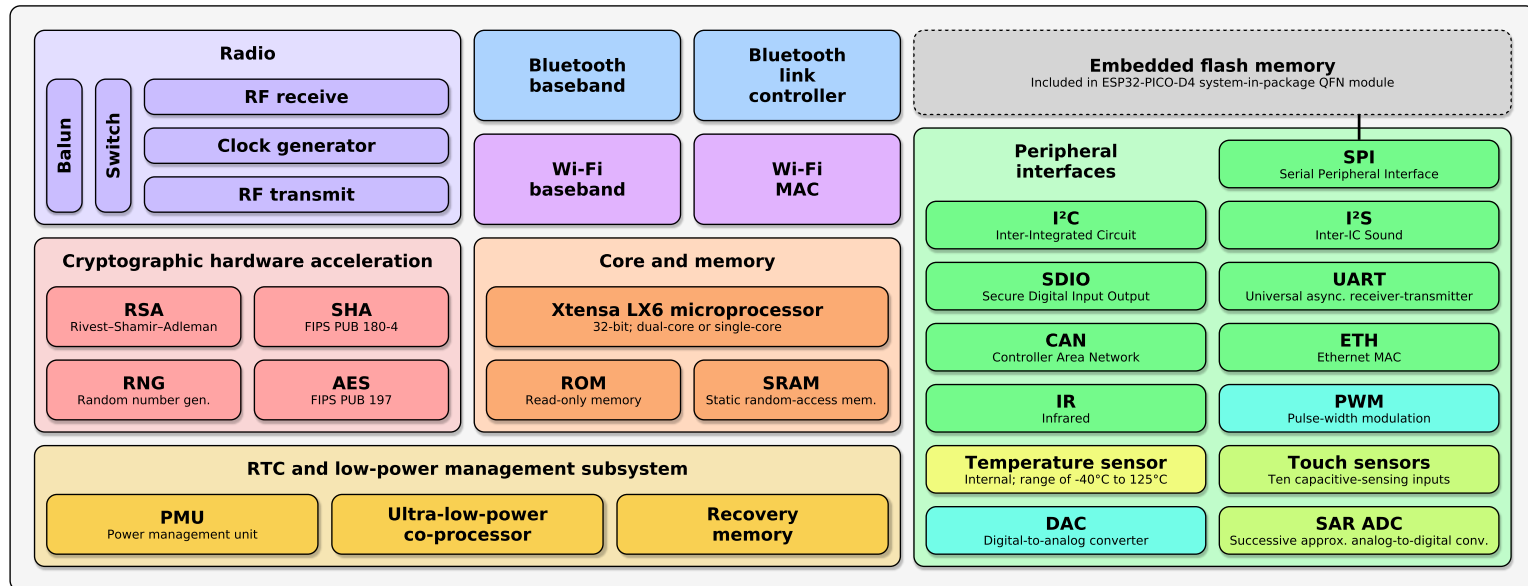
A MORE RECENT VERSION: ESP32

- ESP32 is a series of low-cost, low-power system on a chip microcontrollers with integrated Wi-Fi and dual-mode Bluetooth.
 - successor to the ESP8266 SoC, created and developed by Espressif Systems
- Hardware
 - CPU: Xtensa dual-core (or single-core) 32-bit LX6 microprocessor, operating at 160 or 240 MHz + Ultra low power (ULP) co-processor
 - Memory: 320 KiB RAM, 448 KiB ROM
 - Wireless connectivity: Wi-Fi: 802.11 b/g/n + Bluetooth: v4.2 BR/EDR and BLE (shares the radio with Wi-Fi)
- Peripheral interfaces:
 - 34 × programmable GPIOs, 12-bit SAR ADC up to 18 channels, 2 × 8-bit DACs
 - 4 × SPI, 2 × I²S interfaces, 2 × I²C interfaces
 - 3 × UART

ESP32 ARCHITECTURE

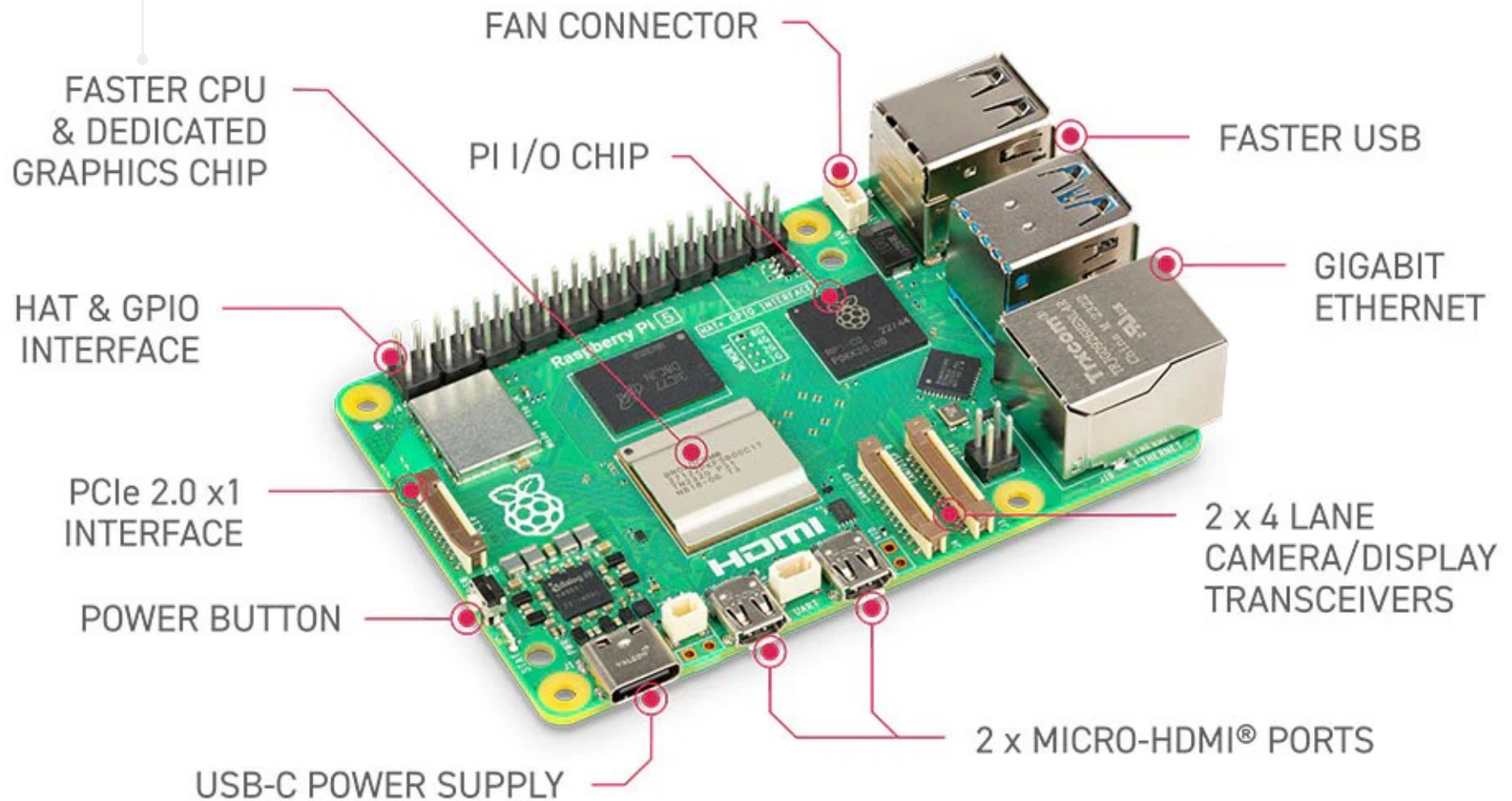


Esspressif ESP32 Wi-Fi & Bluetooth Microcontroller — Function Block Diagram



RASPBERRY PI 5

2.4GHz quad-core, 64-bit Arm Cortex-A76
GPU: 800MHz VideoCore VII GPU
RAM: LPDDR4X-4267 SDRAM (4GB or 8GB)



EMBEDDED OS AND REAL-TIME OS

- Embedded systems based on SoC have enough resources (in terms of computational power and memory) to host a full-fledged operating system (OS)
- *Embedded and Real-Time OS (RTOS)*
 - operating systems designed to be used in embedded systems

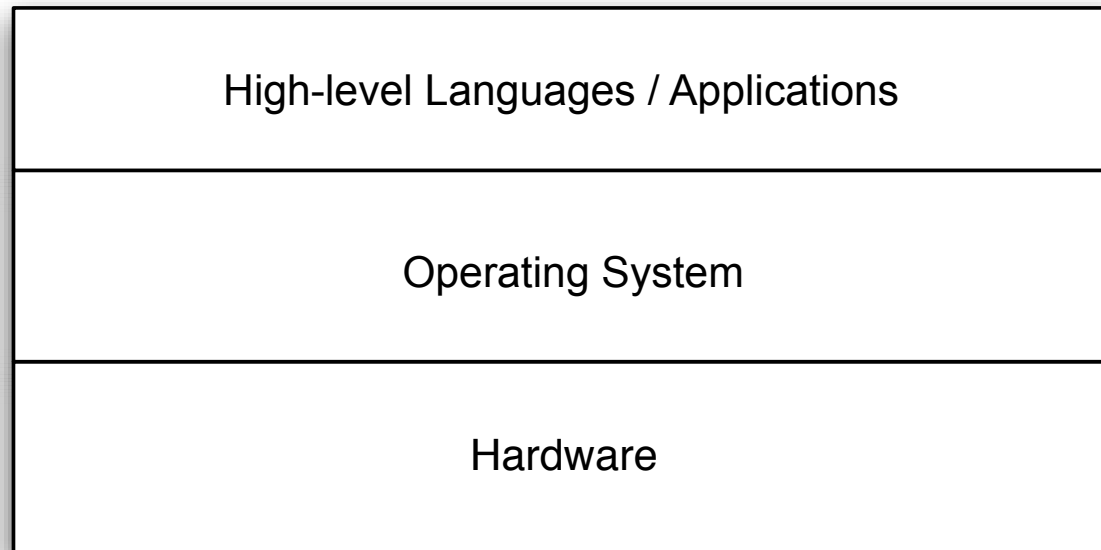
OPERATING SYSTEMS: SOME RECALL

MODERN COMPUTING SYSTEMS: A BIRD'S EYE VIEW

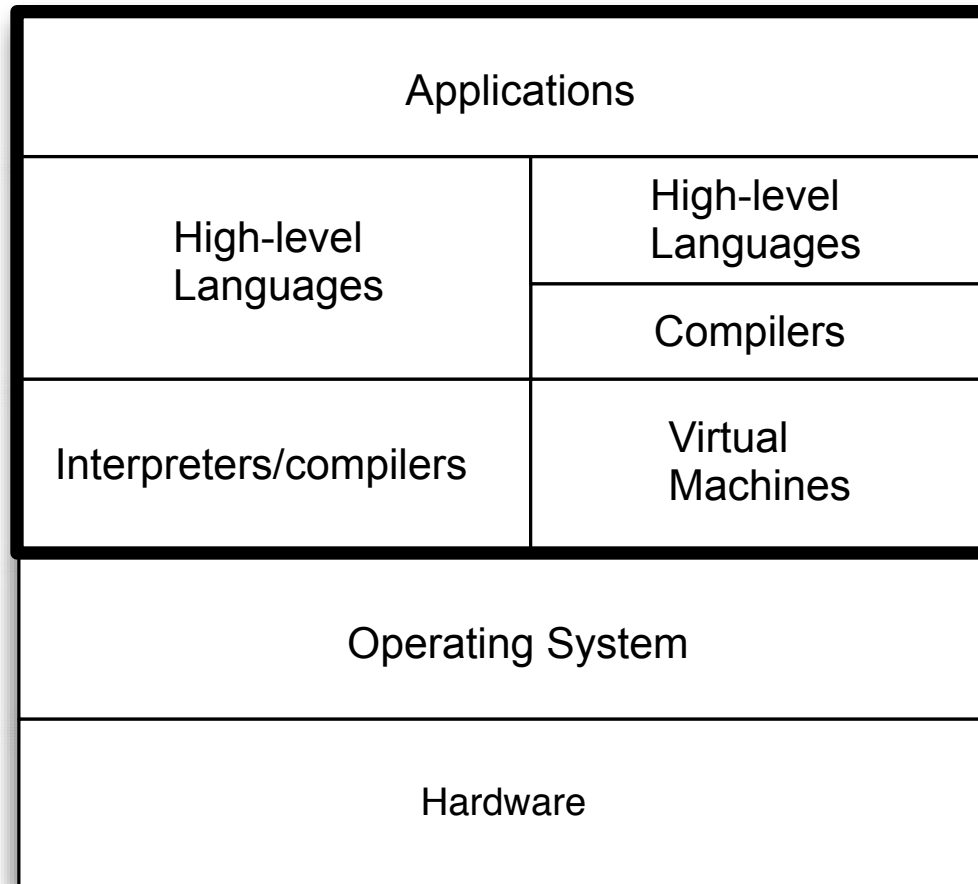
- Computing System
 - software + hardware
 - highest levels: high-level programming languages, applications, complex software systems
 - lowest levels: electronic circuits, logical ports, binary logics
 - design and construction complexity
- Organisation based on **hierarchical levels (layers)**
 - decomposition in modules
 - each module encapsulates some set of functionalities
 - define an abstraction level
 - a module provides an interface to exploit functionalities
 - hide/abstract from implementation details
 - hierarchical organisation (levels or layers)
 - each module corresponds to a layer (level)

LAYERS/LEVELS

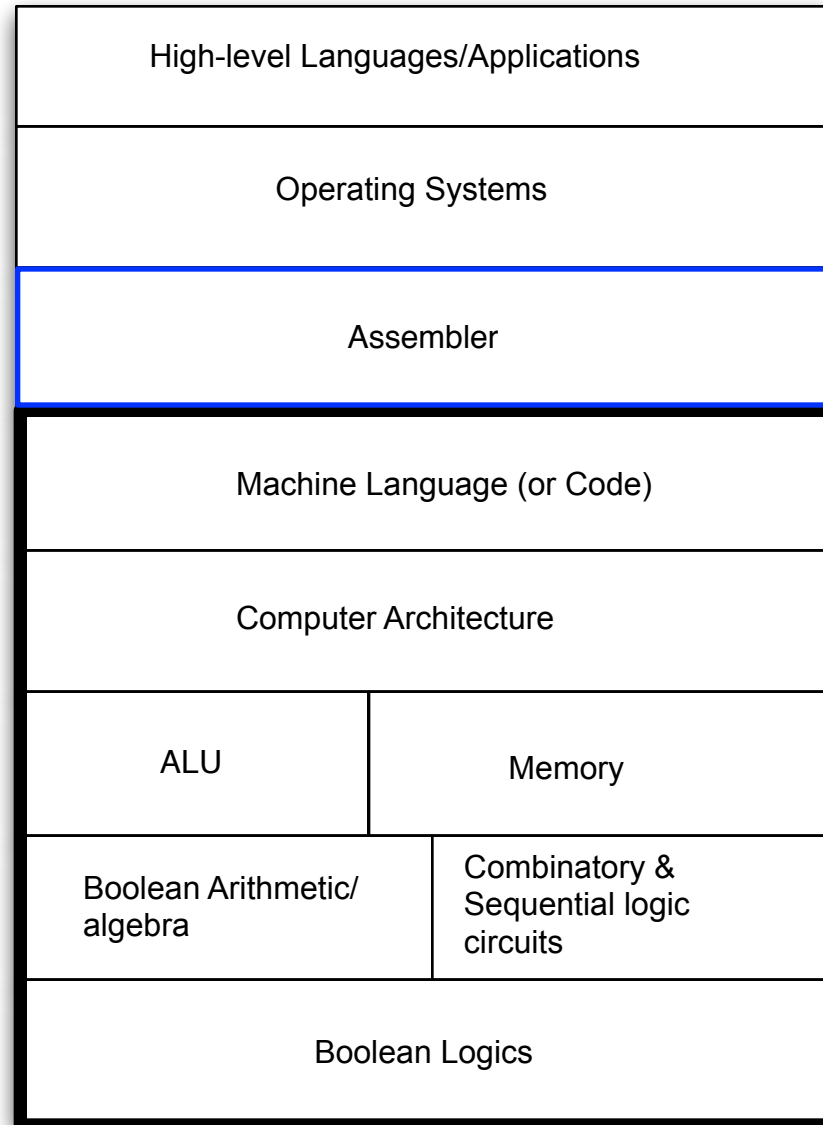
- Highest-level view



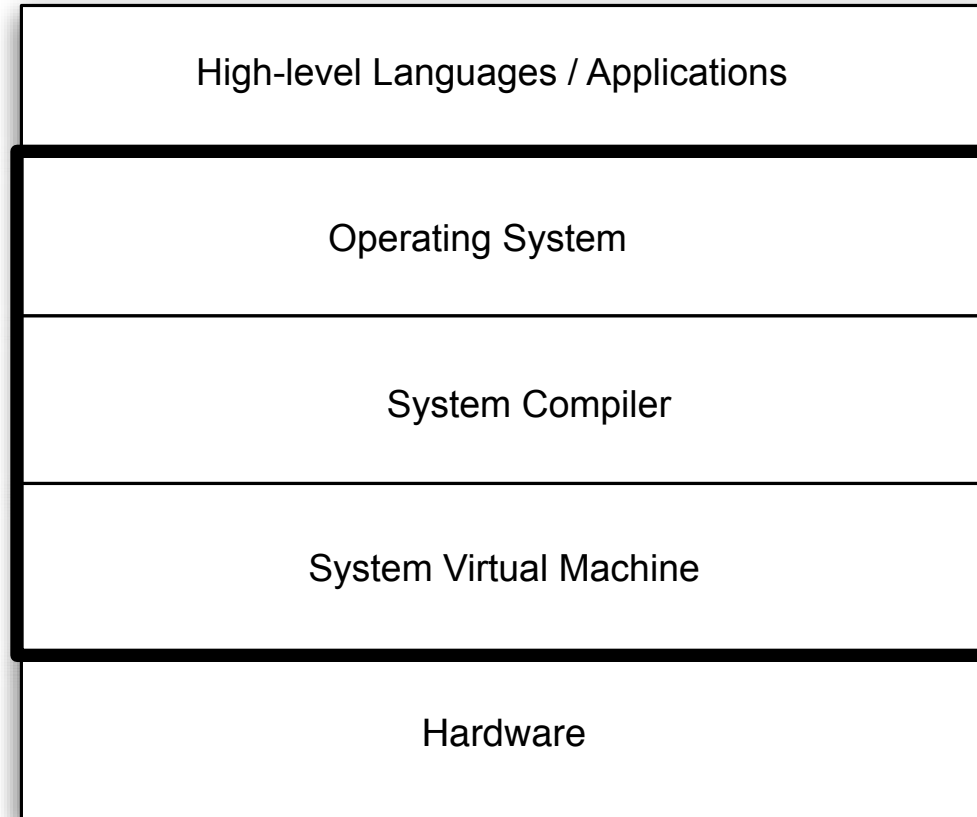
LANGUAGE/APPLICATION LEVEL ZOOM



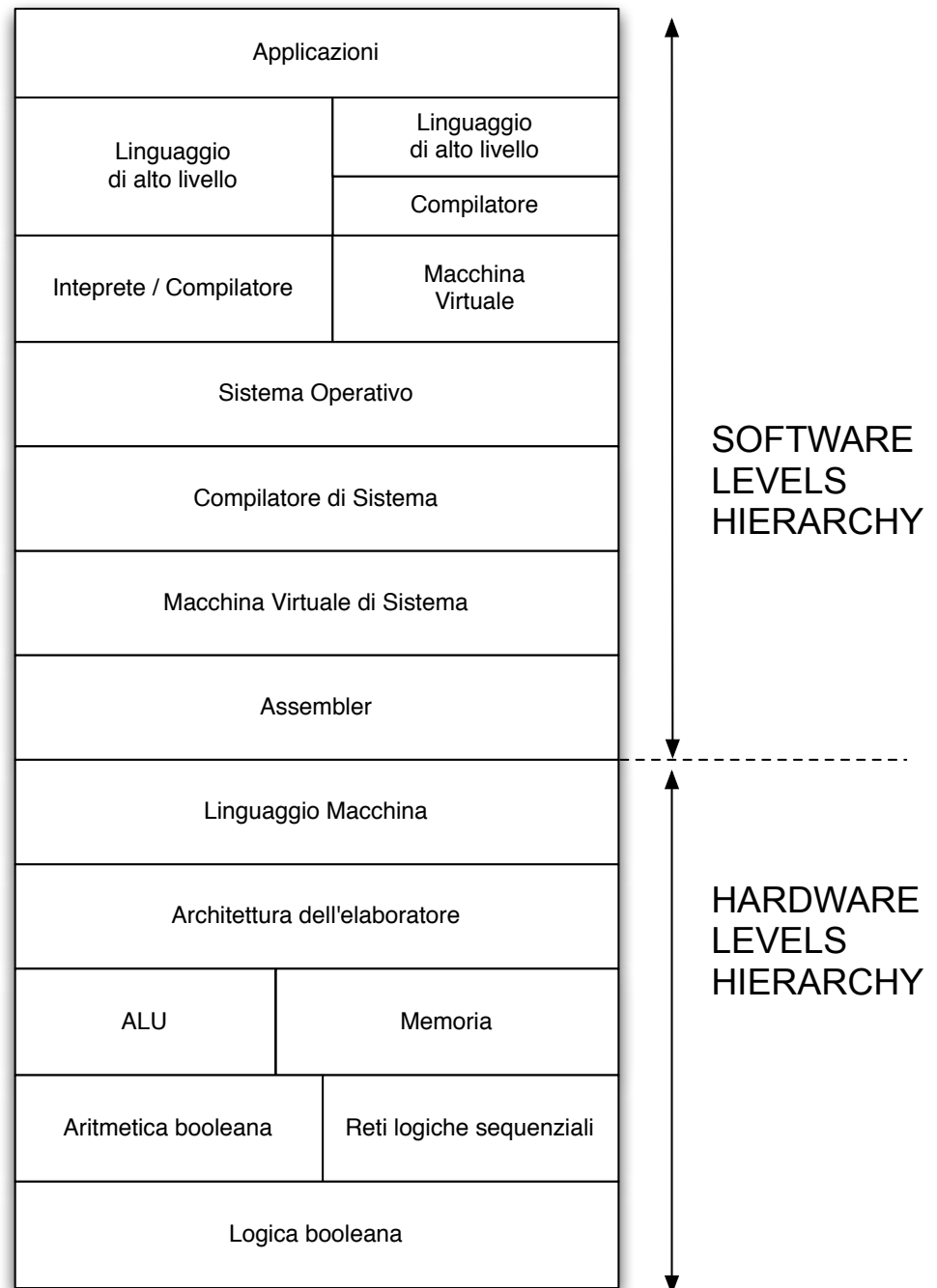
HARDWARE LEVEL ZOOM



VIRTUALISATION LEVEL ZOOM



OVERALL



LEVELS AND INTERFACES

MAIN INTERFACES

ISA

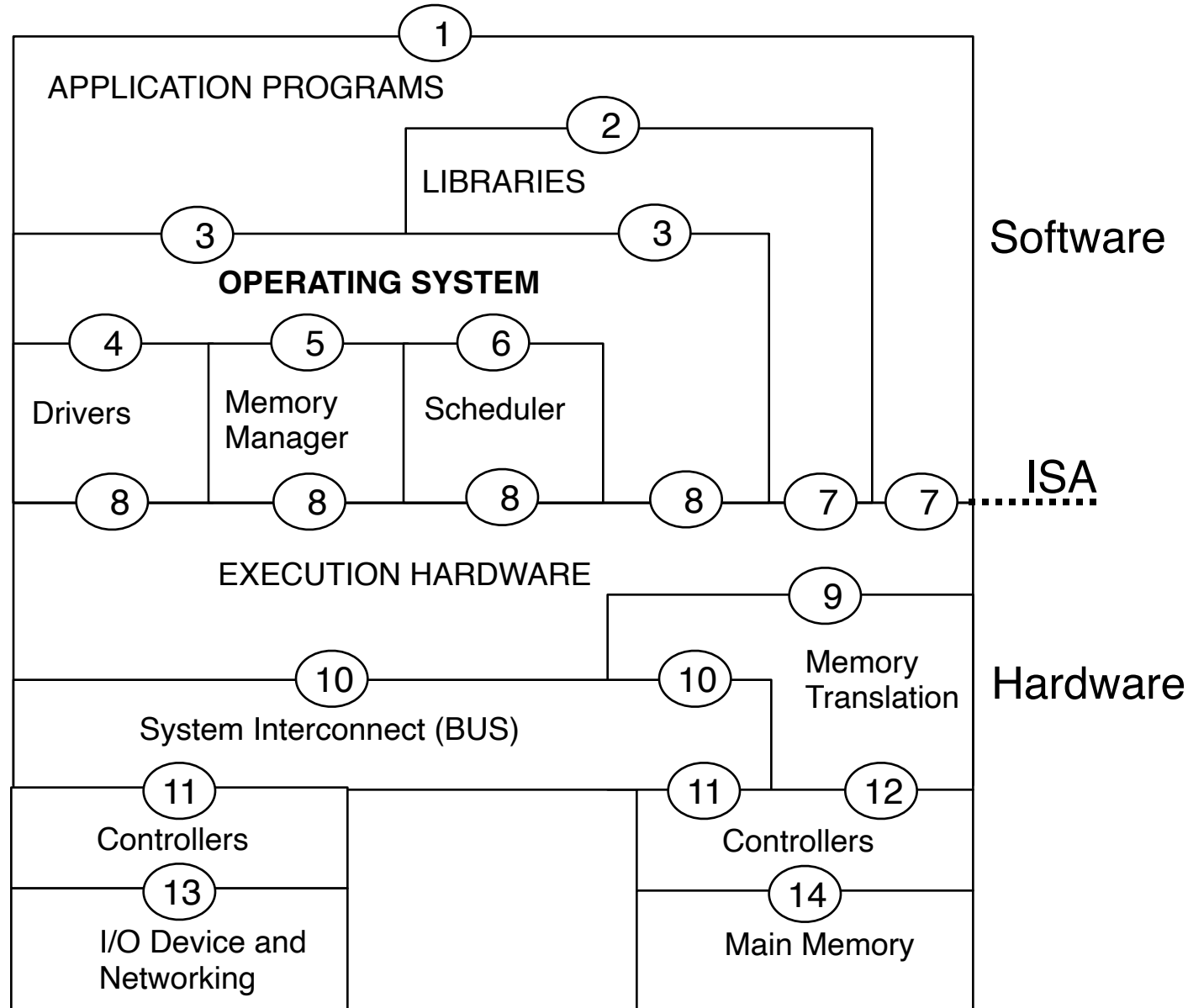
- System ISA: 8
- User ISA: 7 + 8

Application Binary Interface (**ABI**): 7+3

System Call

Interface: 3

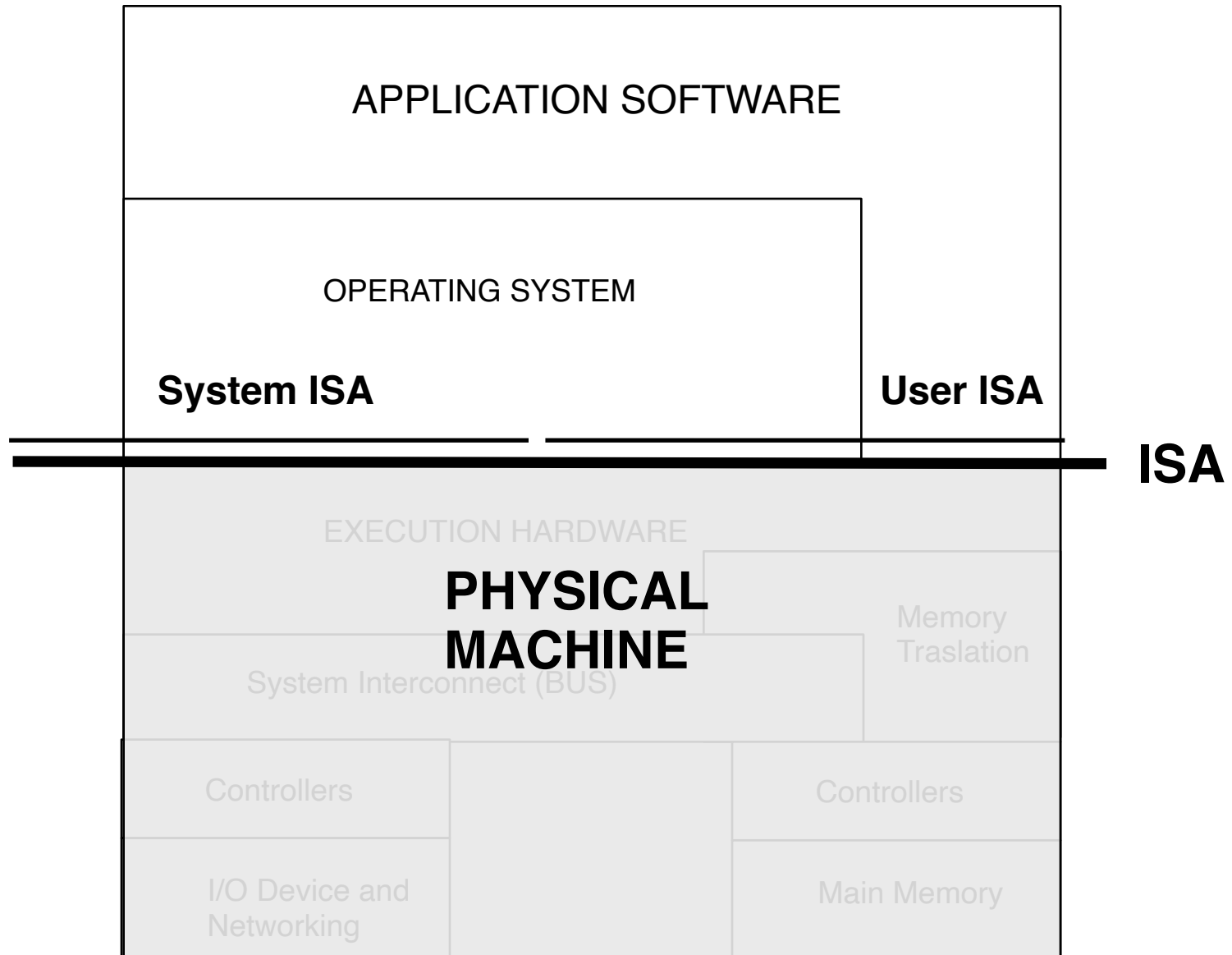
Application Programming Interface (**API**) for High-Level Languages (HLL) (es: C, Java,...): 2



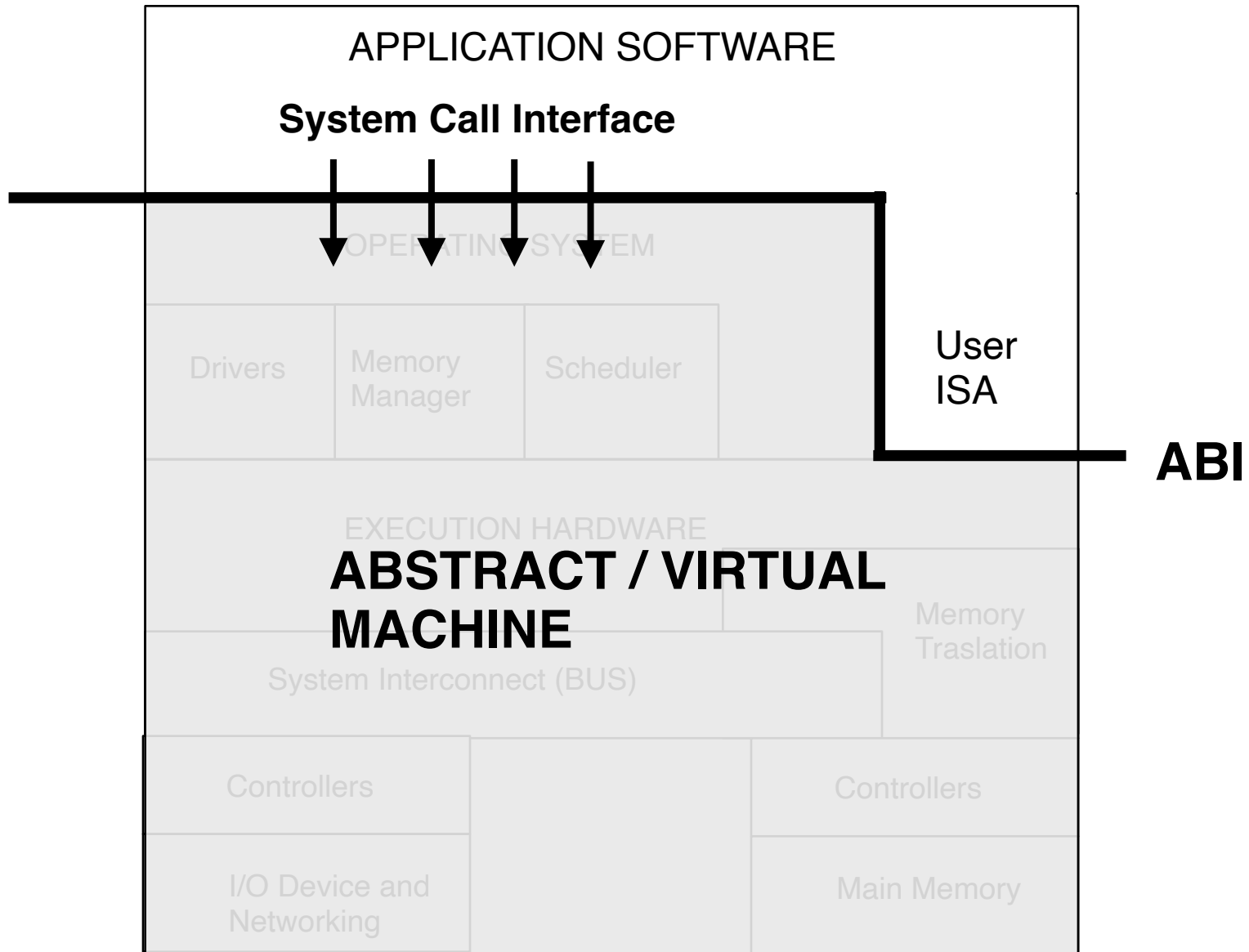
ISA, ABI AND SYSTEM CALL INTERFACE

- Instruction Set Architecture (**ISA**)
 - interface separating HW and SW levels
 - *user ISA*
 - all instructions, aspects, parts that are visible and accessible from a user programs
 - interface 7
 - *system ISA*
 - all instructions, aspects, parts that can be accessed and used only by the OS (es: *privileged* instructions)
 - interface 8
- Application Binary Interface (**ABI**)
 - interface provided by programs to access to HW resources and system services
 - two main components: user ISA + System Call Interface
- System Call interface
 - set of operations that the OS provides to program as fundamental services
 - transferring control to the OS kernel, changing the privilege level

MACHINE INTERFACE: ISA

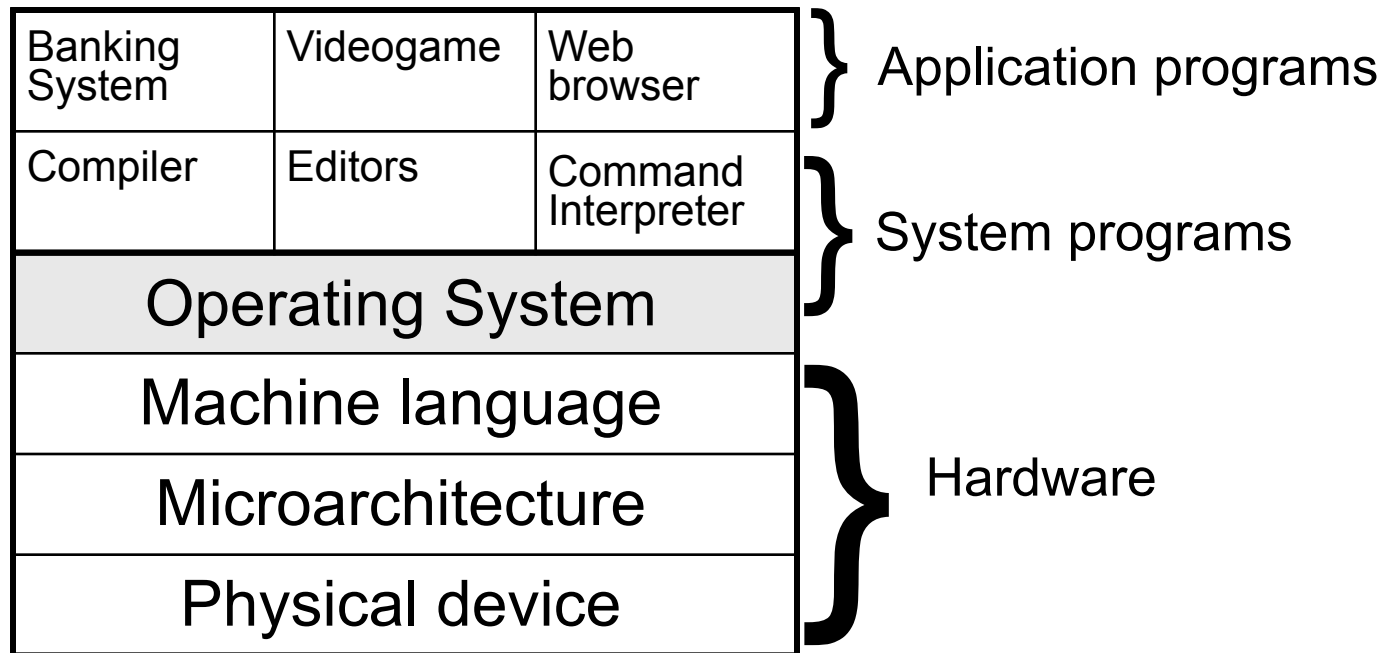


MACHINE INTERFACE: ABI



OPERATING SYSTEM: OVERALL VIEW

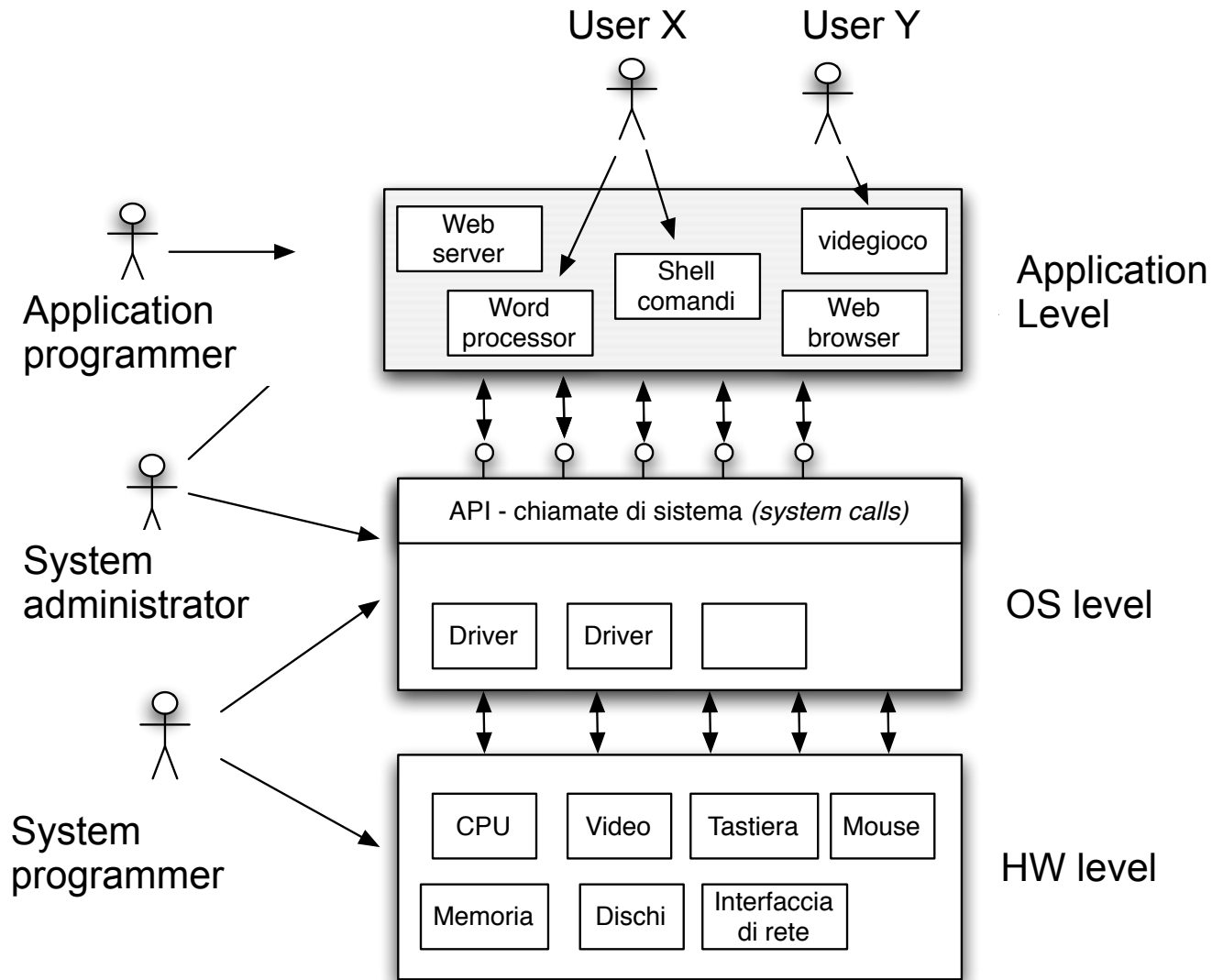
- The **Operating System** (OS) is that software part of a computing system that orchestrates the execution of application programs and functions as intermediary for them to access to the HW



OS: MAIN OBJECTIVES

- Executing user programs and controlling their execution, in particular their access to the resources of the underlying executing machine/
 - OS as an **extended / virtual machine**
 - functions: abstraction, control, protection
- Making the access and use of computer resources effective and efficient
 - OS as a **resource manager**
 - function: optimisation
- Enabling and coordinating the interactions among applications, users and resources
 - concurrent applications
 - multi-user systems

LEVELS AGAIN



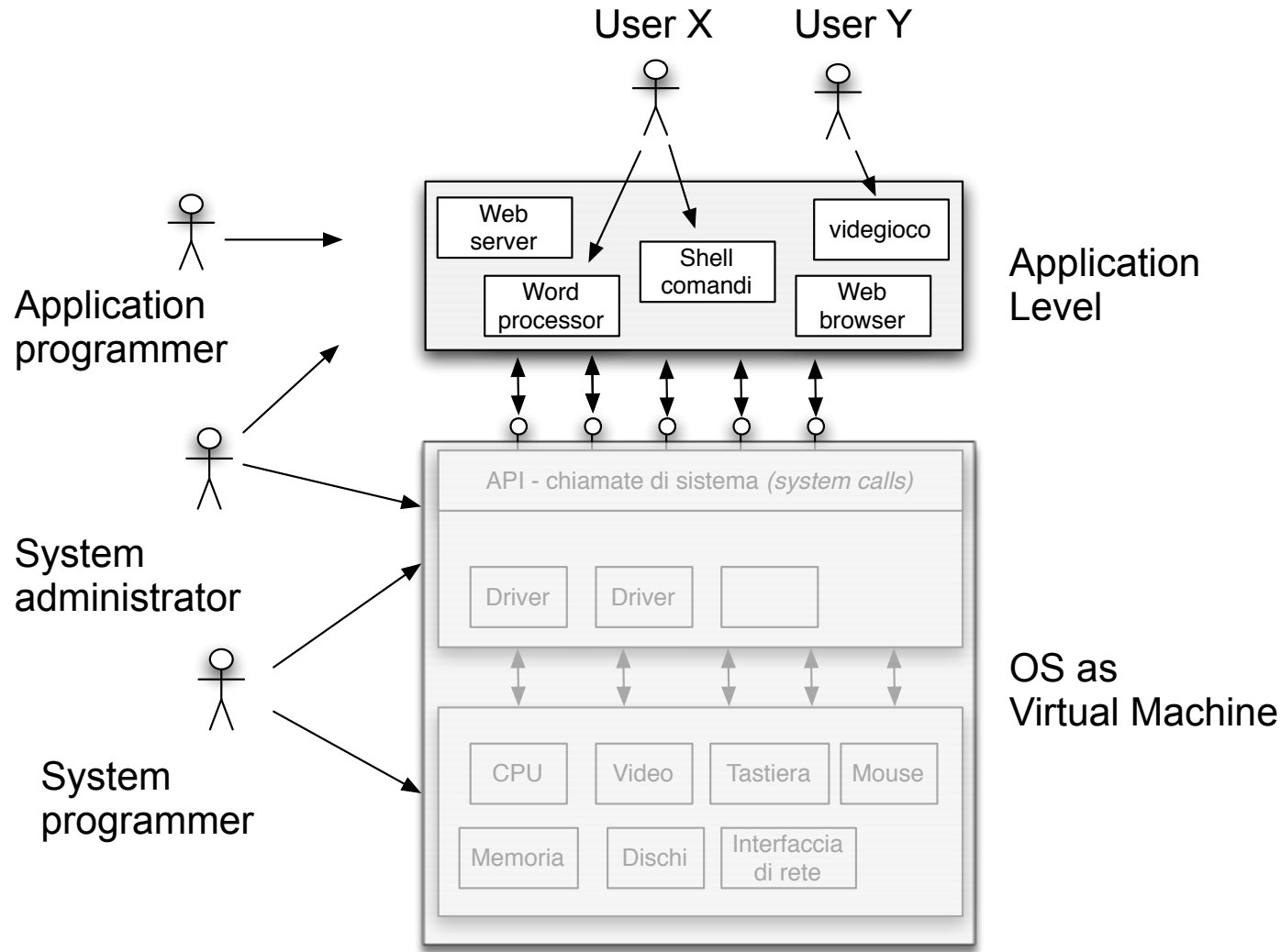
MEDIATION ROLE

- The OS *mediates* the interaction between the application level and the HW level controlling and coordinating the access and use of the HW level as requested by the application, and making the application and HW levels *as much as independent as possible*
 - common functionalities required by applications are factorised into *services* provided by the OS
- Fundamental role of the **API** (Application Programming Interface)
 - programming interface for programs
 - set of functions or OS level primitives referred as **system calls**

OS AS A VIRTUAL MACHINE

- The OS hides the structure of the physical machine to application programs, so that applications see a **virtual or abstract machine**
 - it is a simpler machine than the physical one, providing a higher level of abstraction than the physical layer
 - accessible by means of the system calls
 - they are *the interface* of the virtual machine
 - top-down view

OS AS A VIRTUAL MACHINE



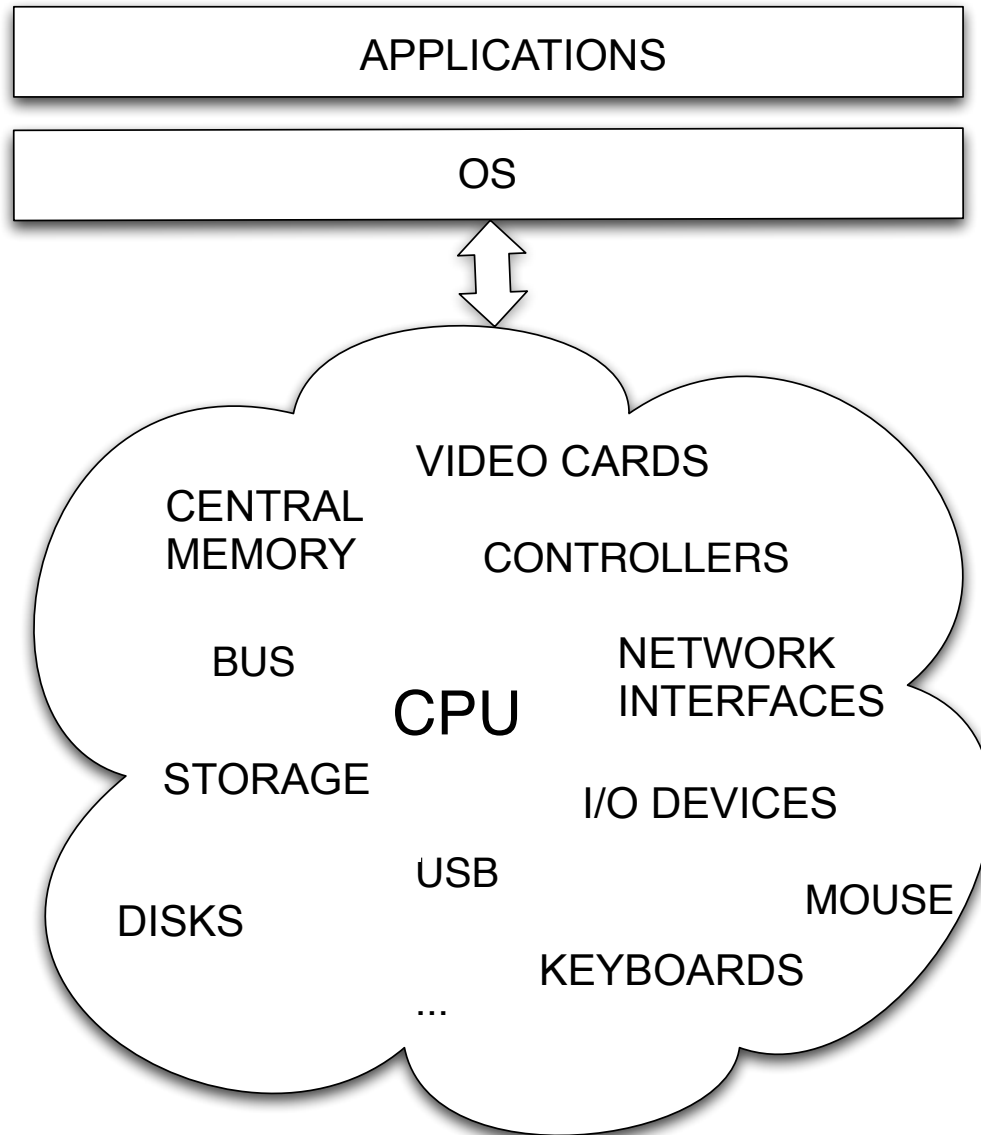
OS AS A VIRTUAL MACHINE - BENEFITS

- To ease the design and development of applications
 - programmers should not necessarily know details about the physical machine to interact with HW resources and can focus on program functionalities
- To ease **porting** of programs
 - the same program can be executed by different virtual machines exploring the same interface, running on different physical machines
 - simplifying *porting* of a program from an OS to another OS

OS AS A RESOURCE MANAGER

- Resource management
 - resources are shared and used by multiple programs in execution
 - possibly *concurrently*
 - possibly belonging to different *users*
 - multi-user systems
- Functions
 - access **optimisation**
 - *scheduling* algorithms
 - **security** and **protection**
 - a program in execution must not create problems to the system or to other programs in execution
 - user data protection and security
- This is the *bottom-up* view
 - from resources (bottom) up to application programs

OS AS A RESOURCE MANAGER



RESOURCE VIRTUALISATION

- **Virtual memory**
 - a program in execution sees the whole memory as a linear memory space, virtually unlimited
 - transparent access (for application programs in execution) to secondary memory/storage as an extension of the primary one
- **Virtual File system**
 - to access and change a file in the file system in spite of the specific type of file systems and their location (local or remote)

PROGRAM EXECUTION

- **Multi-programming**

- capability of loading in central memory multiple programs at the same time and of executing them to optimise CPU usage
- if a program in execution is busy in executing an I/O operation and do not need a CPU, the CPU is allocated to another program in execution
- it allows a first form of **multi-tasking**, i.e. the concurrent execution of multiple programs

- **Time-sharing**

- extension of multi-programming towards the execution of programs that involve the interaction with the user
- capability of executing multiple programs concurrently (in spite of I/O operations), sharing the CPU among programs in execution according to specific scheduling algorithms
- full realisation of multi-tasking

OS/KERNEL FUNCTIONING

- **Interrupt-driven** functioning
 - the execution of the kernel of the OS is triggered by **hardware interrupt** and **software trap**
 - hardware interrupt
 - physical signals sent by devices (es: keyboard, timer, disks,...)
 - **asynchronous**
 - SW trap / interrupt:
 - application programs requests to execute OS services (system calls)
 - or **exceptions**, generated by errors in programs
 - **synchronous**
- Exploiting two distinct execution modalities of CPU
 - **user-mode**
 - when executing user programs
 - **kernel-mode** (super-visor mode, privileged mode, system mode)
 - when executing parts of the kernel

OS MAIN ACTIVITIES

- **Process management**
- **Memory management**
- **Storage management**
 - File System management
 - Disk management
 - caching
- **I/O management**
- **Network management**
- **Protection & security management**
- **Users management**

EMBEDDED AND REAL-TIME OS

EMBEDDED AND REAL-TIME OS

- *Embedded e Real-Time Operating Systems (RTOS)*
 - operating systems designed to be used in embedded systems
- General features
 - compactness
 - efficiency
 - reliability
 - determinism & predictability
 - explained later
- Differently from desktop OS, typically a RTOS is designed *to run **one** application at a time*
 - the application is typically multi-task/multi-threaded

EXAMPLES

- Open-source
 - FreeRTOS, eCos
 - RT Linux, Linux Embedded
- Commerciali
 - VxWorks, LynxOS, Micrium μ C/OS-III
 - QNX Neutrino
 - Windows CE
- Una lista
 - https://en.wikipedia.org/wiki/Comparison_of_real-time_operating_systems

EXAMPLE: μC/OS FAMILY

(1) The port developer is responsible for providing the μC/OS-III CPU Specific portion. A μC/OS-III port consists of writing or changing the contents of four kernel-specific files: `os_cpu.h`, `os_cpu_a.asm`, `os_cpu_a.inc` and `os_cpu_c.c`.

(2) A port also involves writing or changing the contents of two CPU specific files: `cpu.h` and `cpu_a.asm`. `cpu_core.c` is generally generic and should not require modifications.

(3) A Board Support Package (BSP) is generally necessary to interface μC/OS-III to a timer (which is used for the clock tick) and an interrupt controller.

(4) Some semiconductor manufacturers provide source and header files to access on-chip peripherals. These are contained in CPU/MCU specific files. You generally don't need to modify any of these and thus, you can use them as-is.

Configuration Files

```
cpu_cfg.h
lib_cfg.h
os_cfg.h
os_cfg_app.h
```

Application Code

```
app.c
app.h
app_cfg.h
```

μC/OS-III CPU Independent

```
os_cfg_app.c
os_type.h
os_core.c
os_dbg.c
os_flag.c
os_int.c
os_mem.c
os_msg.c
os_mutex.c
os_pend_multi.c
os_prio.c
os_q.c
os_sem.c
os_stat.c
os_task.c
os_tick.c
os_time.c
os_tmr.c
os_var.c
os.h
```

μC/LIB Libraries

```
lib_ascii.c
lib_ascii.c
lib_def.h
lib_math.c
lib_math.h
lib_mem_a.asm
lib_mem.c
lib_mem.h
lib_str.c
lib_str.h
```

μC/OS-III CPU Specific

```
os_cpu.h
os_cpu_a.asm
os_cpu_a.inc
os_cpu_c.c
```

(1)

μC/CPU CPU Specific

```
cpu.h
cpu_bsp.c
cpu_def.h
cpu_a.asm
cpu_c.c
cpu_core.c
cpu_core.h
```

(2)

BSP

Board Support Package

```
bsp.c
bsp.h
bsp_int.c
bsp_int.h
```

(3)

CPU

```
*.c
*.h
```

(4)

Software/Firmware

Hardware

CPU

Timer

Interrupt
Controller

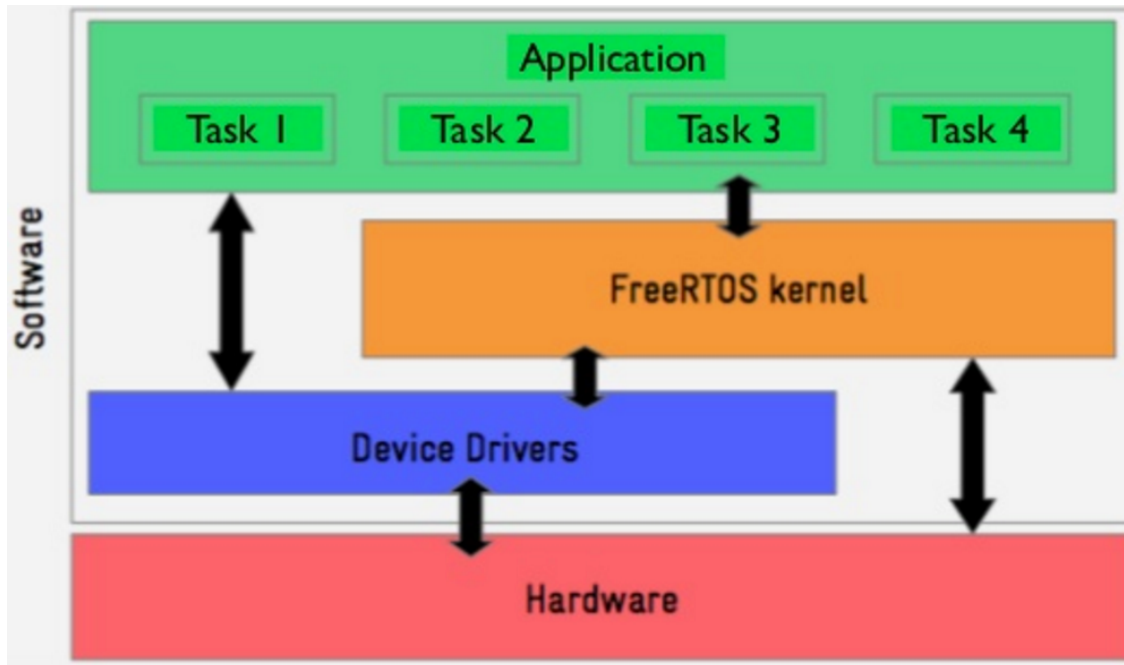
<https://doc.micrium.com/>

μC/OS-II and μC/OS-III Feature Comparison Chart

EXAMPLE: μC/OS FAMILY

Feature	μC/OS-II	μC/OS-III
Preemptive Multitasking	Yes	Yes
Maximum number of tasks	255	Unlimited
Number of tasks at each priority level	1	Unlimited
Round Robin Scheduling	No	Yes
Semaphores	Yes	Yes
Mutual Exclusion Semaphores	Yes	Yes (Nestable)
Event Flags	Yes	Yes
Message Mailboxes	Yes	No (not needed)
Message Queues	Yes	Yes
Fixed Sized Memory Management	Yes	Yes
Signal a task without requiring a semaphore	No	Yes
Send messages to a task without requiring a message queue	No	Yes
Software Timers	Yes	Yes
Task suspend/resume	Yes	Yes (Nestable)
Deadlock prevention	Yes	Yes
Scalable	Yes	Yes
Code Footprint	6K to 26K	6K to 20K
Data Footprint	1K+	1K+
ROMable	Yes	Yes
Run-time configurable	No	Yes
Compile-time configurable	Yes	Yes
ASCII names for each kernel object	Yes	Yes
Interrupt Latency (Cortex-M3)	120~	< 100~
Pend on multiple objects	Yes	Yes
Task registers	Yes	Yes
Built-in performance measurements	Limited	Extensive
User definable hook functions	Yes	Yes
Time stamps on posts	No	Yes
Built-in Kernel Awareness support	Yes	Yes
Optimizable Scheduler in assembly language	No	Yes
Tick handling at task level	No	Yes
Source code available	Yes	Yes

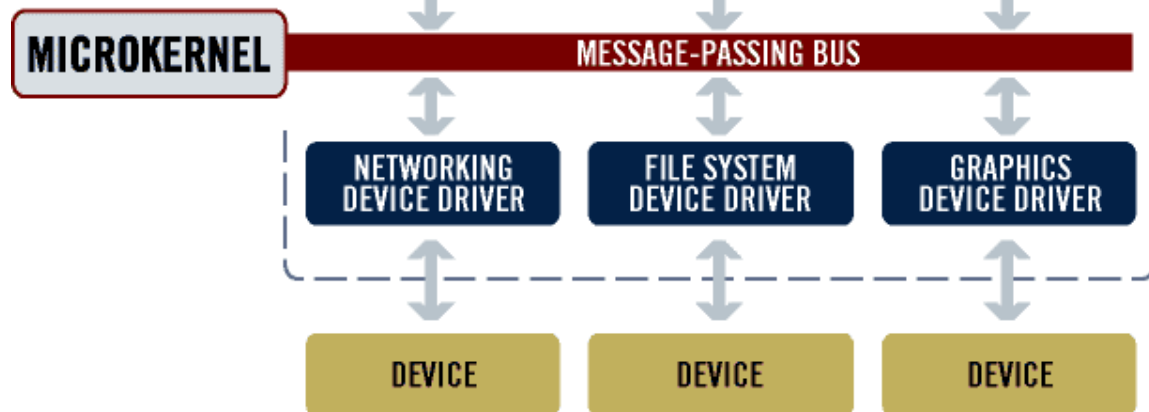
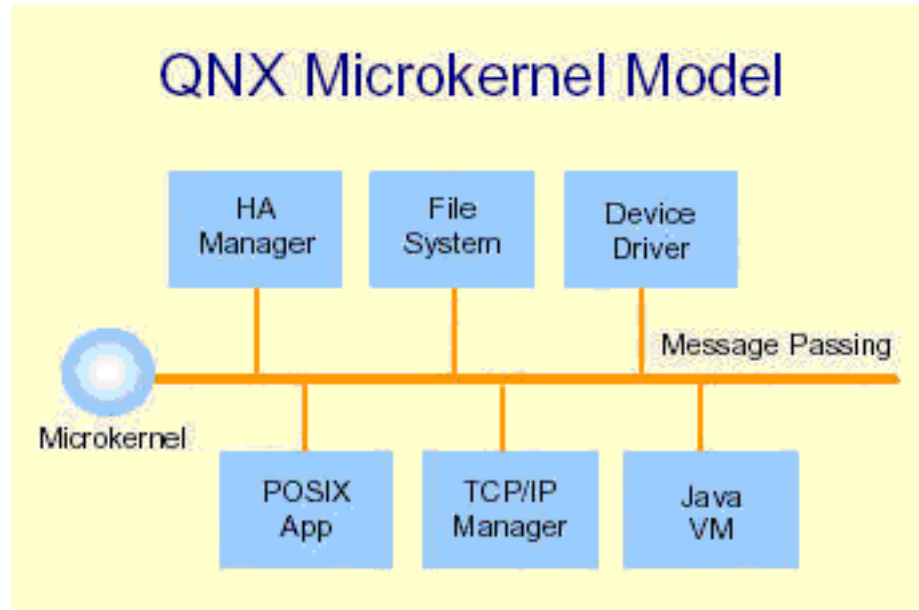
EXAMPLE: FreeRTOS



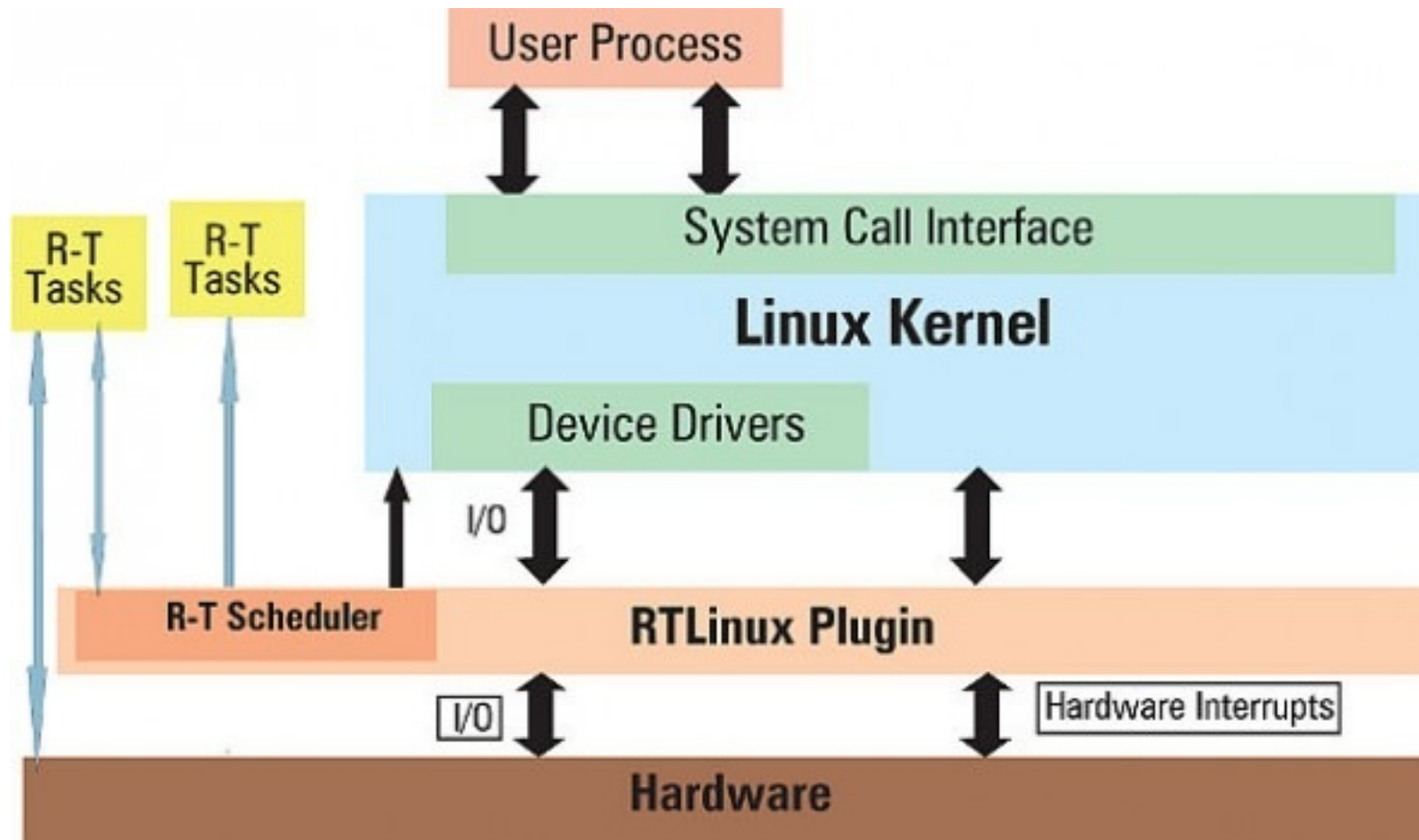
FreeRTOS User Tasks and ISR Code
FreeRTOS Hardware-Independent Code
FreeRTOS Hardware-Dependent Code
Hardware

<https://www.freertos.org/>

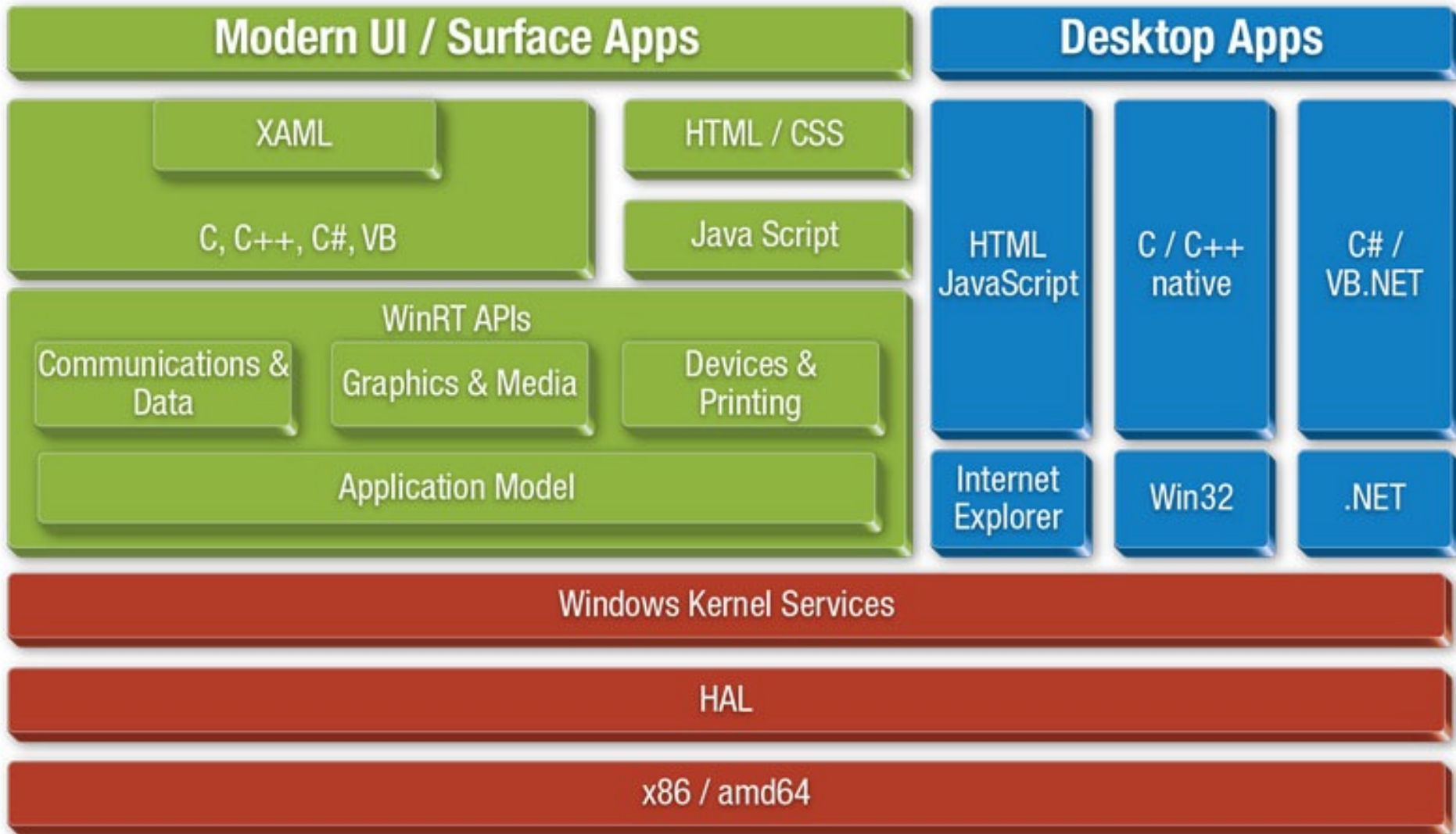
EXAMPLE: QNX REAL-TIME OS



RT-LINUX ARCHITECTURE (REAL-TIME LINUX)

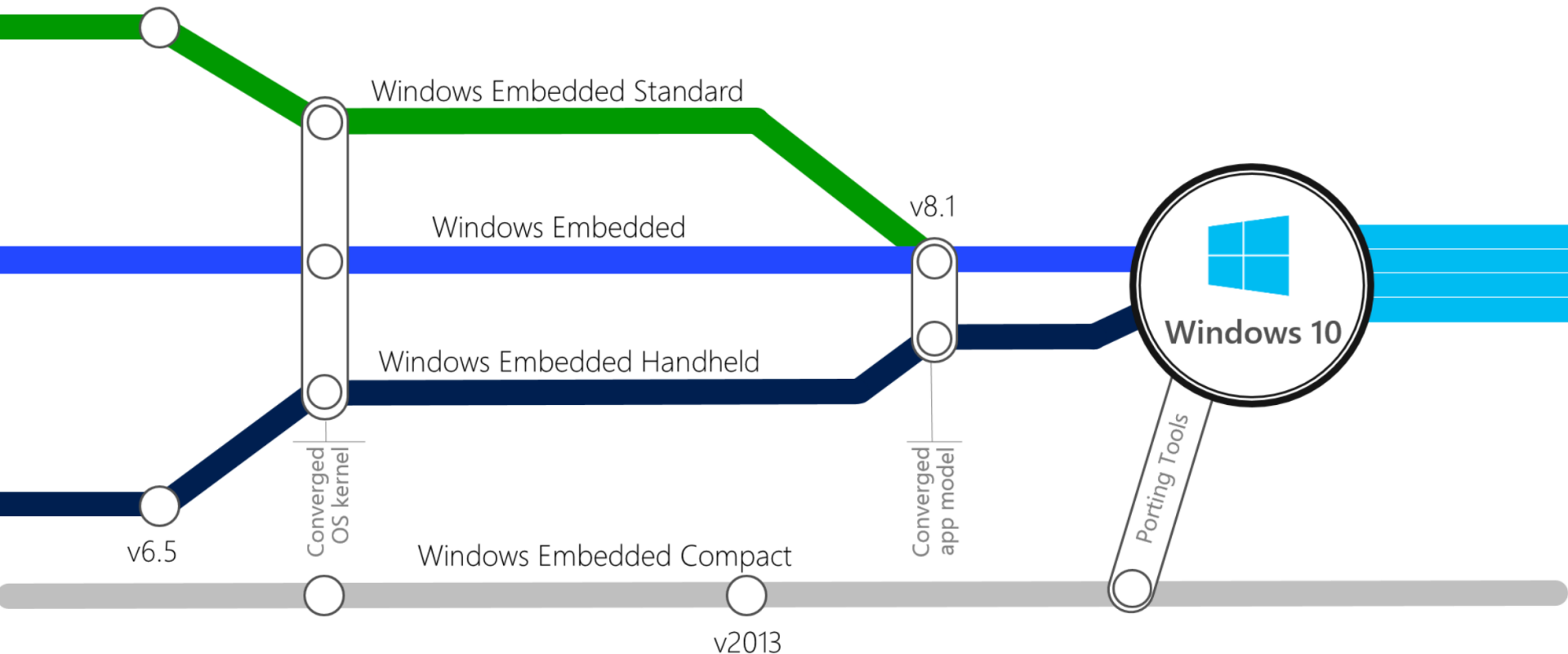


WINDOWS 8 EMBEDDED



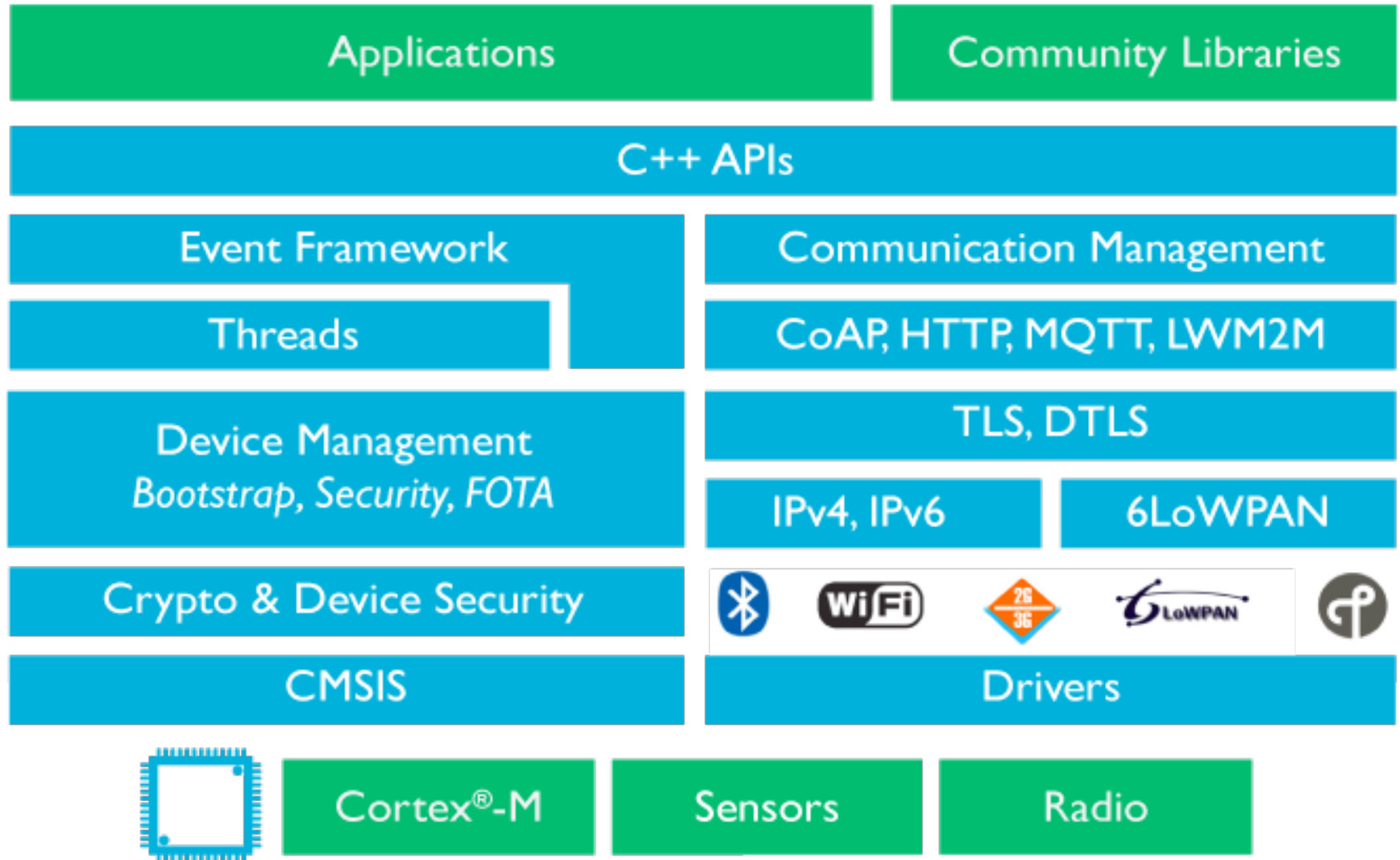
WINDOWS IOT CORE

IoT Platform Convergence



<https://docs.microsoft.com/en-us/windows/iot-core/windows-iot>

ARM mbed IoT OS ARCHITECTURE



REAL-TIME EMBEDDED SYSTEMS

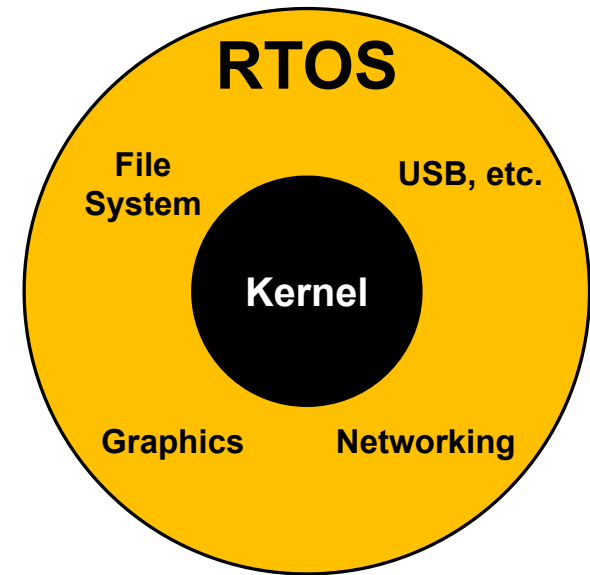
- Real-time
 - they must react and manage events and inputs within specified time intervals
 - they manage the execution of tasks/computations that have **deadlines**
- Subclasses
 - **hard** real-time
 - deadlines are correctness requirements, they must be satisfied (otherwise: failures)
 - examples: safety-critical systems
 - **soft** real-time
 - deadlines should be respected in normal conditions
 - can be violated in some cases and the violation is meant to be managed

REAL-TIME SYSTEMS & DETERMINISM

- **Determinism** is the most important aspect of real-time systems
 - hard real-time, in particular
- Determinism in this case means that the system should be **predictable**:
 - the time needed to perform some task
 - the maximum time required to execute some action or getting some input or from a sensor or to react to some interrupt
 - the number of cycles required to exec some operation must be always the same
- In these systems, the execution can be interrupted (by means of interrupts), however the overhead (interrupt latency, context switch, interrupt handler execution time) should be known

RTOS ORGANISATION

- Structure similar to normal OSs
 - kernel
 - file system
 - networking
 - USB
 - Graphics
- **Kernel tasks**
 - task scheduling and management
 - inter-task interaction & synchronisation
 - semaphores
 - queues / mailboxes — msg passing
 - timer management
 - I/O device management
 - memory management
 - interrupt management & signals



CPU execution modalities

- *kernel* mode
- *user* mode

RTOS BENEFITS

- Improving responsiveness and reducing overhead
- Simplifying resource sharing
- Simplifying development, debugging, maintenance
 - for complex embedded system
- Enhancing portability
- Exploiting proper service layer & middleware
- Making application time-to-market quicker

RESPONSIVENESS AND OVERHEAD

- Programs based on systems with no RTOS are typically based on the super-loop architecture, adopting polling as general strategy to verify conditions and triggering functions and actions or cooperative schedulers for executing tasks => different kinds of problems
 - reduced efficiency
 - weakness of cooperative schedulers
 - ..
- An RTOS allows for:
 - avoiding polling/looping by exploiting the possibility to execute a "context switch" from a task to another one, at a processor level
 - support for pre-emptive multi-tasking
 - maximise the CPU time for applications, minimising the overhead of task management

RESOURCE MANAGEMENT

- An RTOS allows tasks to share resources and provides functionalities to manage and control their access
 - dynamic memory allocation/deallocation
 - semaphores/mutex to control the access to HW resource or realise critical sections
 - mechanisms to avoid problems due to interferences between priorities and critical sections
 - e.g. the priority inversion problem (described later)

SIMPLIFYING DEVELOPMENT AND DEBUGGING

- From an application perspective, an RTOS provides an abstraction layer that simplify the access to the HW
 - the development of applications does not require the access and management of low-level HW aspects (interrupts, timers, ports, etc.)
- RTOS interface: system calls
- Specific HW+SW support for debugging
- Modular development of an application

HIGH-LEVEL SERVICES & MIDDLEWARE

- An RTOS enables the use of frameworks/platforms/middleware that provide services, at different levels
- Examples
 - File system
 - TCP/IP network stack
 - USB stack
 - Graphics

PORTABILITY AND MAINTENANCE

- An application using the RTOS API does never access directly to the HW
 - this makes it possible to change the underlying HW in a transparent way
 - the application can be executed on any HW supported by the RTOS

WHEN AN RTOS IS NOT USEFUL

- Simple super-loop applications
- Single-purpose / single-task applications
 - typically with a dimension smaller than 32 KiB

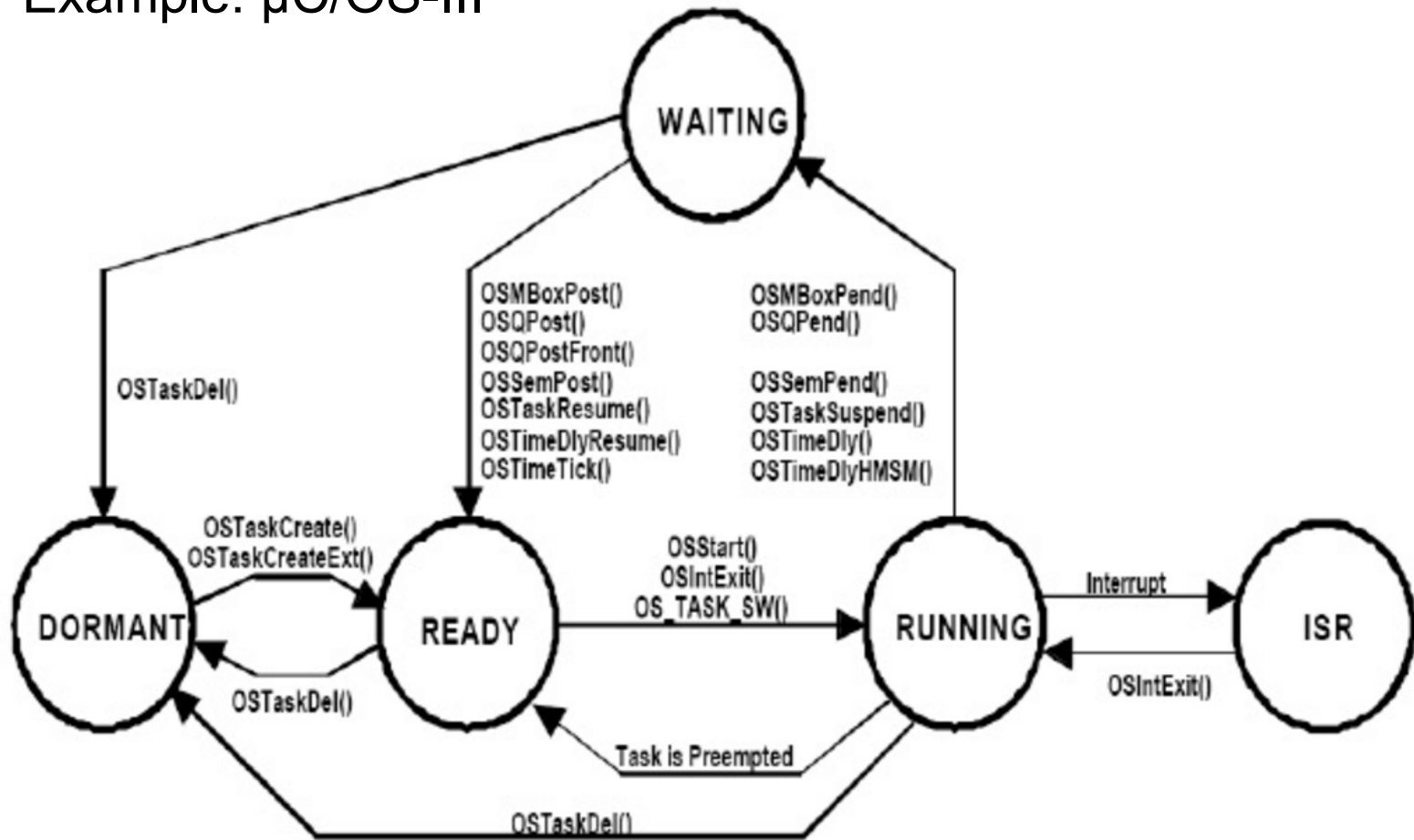
ABOUT TASKS AND SCHEDULING IN RTOS

PROCESSES, TASKS AND THREADS

- Process (definition)
 - a program in execution
 - has its own memory
 - can have multiple threads
- Thread
 - an independent thread of control or control flow
 - threads part of the same process share the process memory
- Multi-threading: concurrent execution of multiple threads
 - threads: often called **tasks** in RTOS
 - a way for modularising an application
 - notion of *priority*
- Scheduling done by the kernel
 - to realise multi-tasking, multi-threading
 - various strategies and algorithms
 - pre-emptive vs. cooperative
 - round-robin + priority-based

TASK STATES

- Example: μ C/OS-III



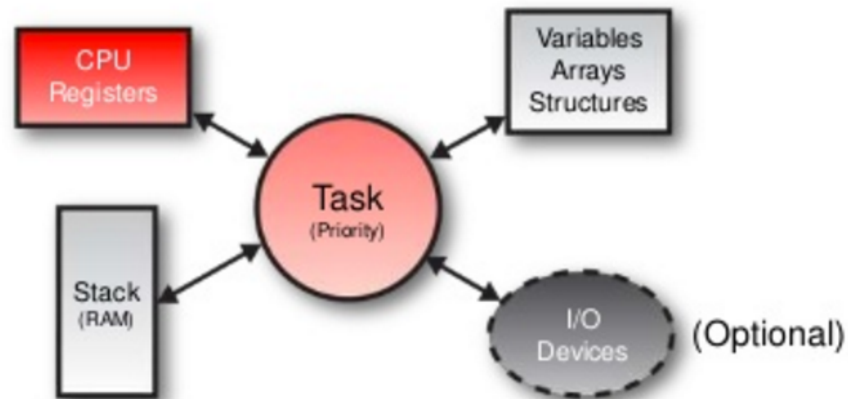
CONTEXT SWITCH

- Thread/task context
 - the information necessary for restarting the execution of a thread when a context switch occurred
 - content of processor registers, program counter, stack pointer
 - saved when a thread/task is preempted
 - restored when a thread/task is resumed
- Context switch
 - interruption of the execution of current thread
 - control given to the kernel and eventually to another thread
 - has a cost

TASK PROGRAMMING

- Example: μ C/OS-III

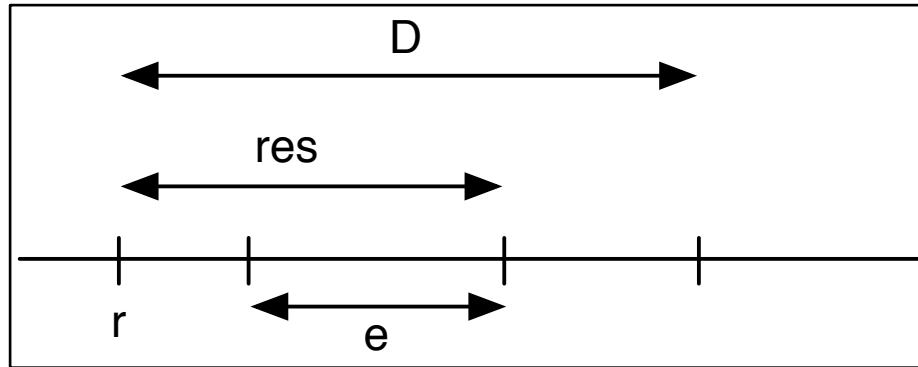
```
void MyTask (void *p_arg)
{
    Do something with 'argument' p_arg;
    Task initialization;
    for (;;) {
        Wait for event;      /* Time to expire ... */
                           /* Signal/Msg from ISR ... */
                           /* Signal/Msg from task ... */
        /* Processing (Your Code) */
    }
}
```



TASK COMMUNICATION AND SYNCHRONISATION MECHANISMS

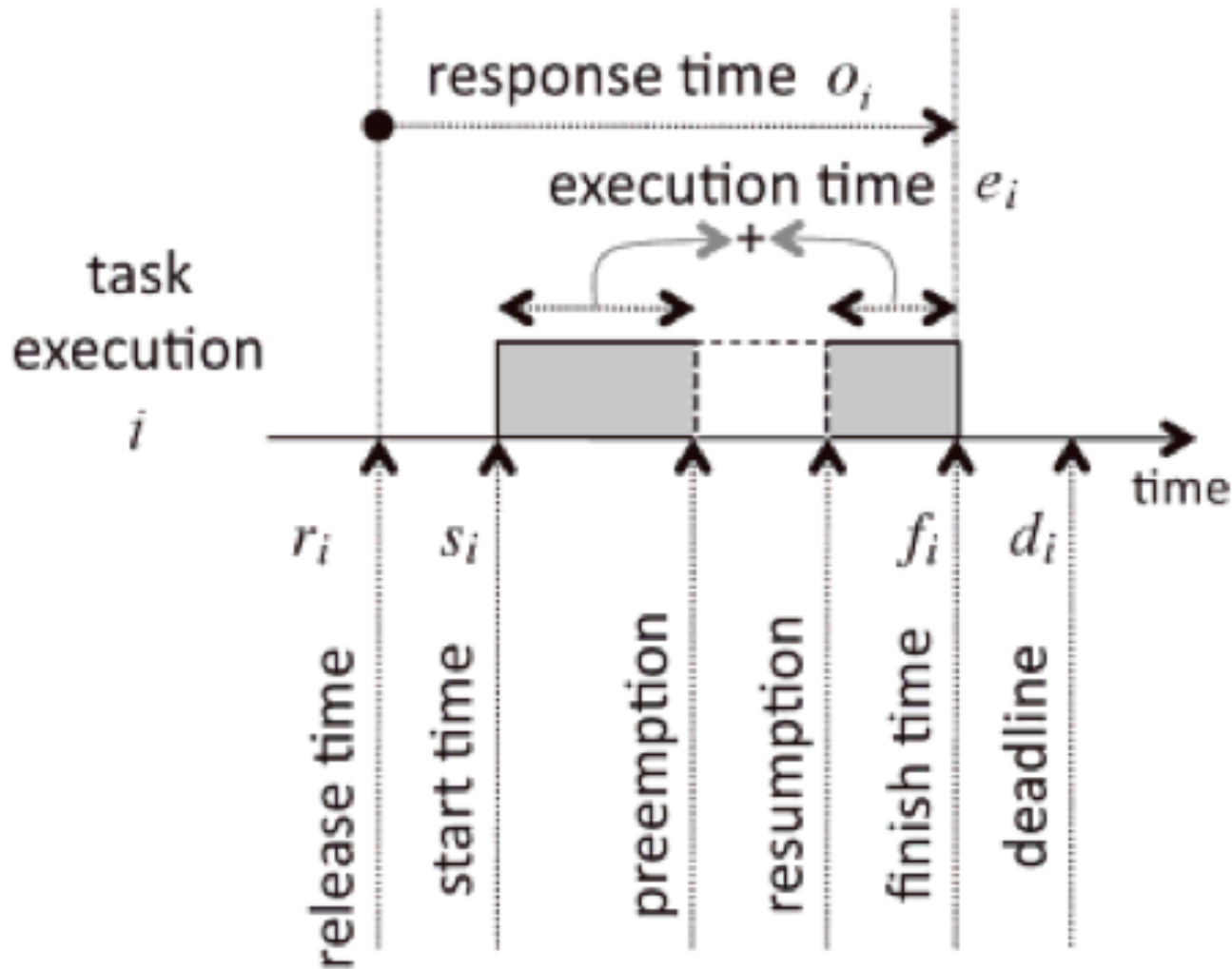
- Semaphores
- Communication based on message passing
- Event-based architectures

TASKS: TEMPORAL PARAMETERS



- Temporal properties of tasks are defined by 4 fundamental parameters
 - **release time: r**
 - the time point in which the task enters the ready queue, being ready to be executed
 - **execution time: e (= WCET)**
 - maximum duration of the task
 - **response time: res**
 - interval time between release of the task and its completion
 - **deadline: D**
 - maximum time allows for executing the task

TASKS: TEMPORAL PARAMETERS



PERIODIC AND APERIODIC TASKS

- Task can be:
 - **periodic**
 - the task is released at a fixed temporal interval (period)
 - **aperiodic**
 - the task is released at arbitrary times
 - **sporadic**
 - like aperiodic, but with hard deadlines
- Designers must decide what to do when a deadline is not satisfied
 - a task must continuous its execution (so delaying the other tasks)? Or, its termination should be forced?

SCHEDULING FAMILIES

- *Big loop* scheduling (i.e. cooperative scheduling in a super-loop)
 - each task is *polled* to check if it should be executed
 - polling goes on either sequentially or following a priority order
 - not efficient, low responsiveness
- Round-robin scheduling
 - the processor is allocated turn by turn to ready tasks (threads)
 - cooperative scheduling: the threads are executed up to their completion or a blocking call
 - preemptive scheduling: each thread executes for a time slice + preemption
- **Priority-based** scheduling
 - the processor always executes the thread with the highest priority
 - for threads with the same priority => round-robin
 - preemption
 - if there is ready thread with higher priority => this is executed

PRIORITY SCHEDULING

- All RTOSs support priority-based scheduling
 - possibly integrated with round-robin
- Advantages
 - max responsiveness
- Problems
 - complexity and cost of pre-emption
 - possibility of starvation
 - inversion priority problem

SCHEDULING IN SYNCHRONOUS AND ASYNCHRONOUS REAL-TIME SYSTEMS

- Two main categories of real-time systems, from a scheduling point of view
 - synchronous RT systems
 - strongly related to synchronous FSM and to the task scheduling discussed in module 2.3
 - asynchronous RT systems
 - strongly related to asynchronous FSM

SYNCHRONOUS RT SYSTEMS

- A hardware clock is used to subdivide CPU time in temporal intervals called **frames**
 - similar to the period in sync FSM and period of the scheduler in module 2.3
- A program must be subdivided into tasks so that each task can be completely executed in single frame in the worst case
- Simple scheduling
 - a *scheduling table (list)* keeps track of the tasks to be execute in each frame
 - if the execution of a task would take more than a frame, then the task must be divided in sub-tasks, to be individually executed in multiple frames
 - frame duration is a very important parameter to choose when designing the system
- Approach typically used in **hard-real time** systems

ASYNCHRONOUS RT SYSTEMS

- In asynchronous RT systems, a full-fledged pre-emptive priority-based scheduler is used
 - tasks are scheduled depending on their priority
 - tasks with the same priority are scheduled using round-robin pre-emptive scheduling
- Scheduler implementation is based on *scheduling events* that invoke the scheduler
 - scheduling events are generated:
 - when a task is enqueued in the ready-queue
 - by the timer, at a some frequency, in order to keep track of time passing and realise round-robin pre-emption

ASSIGNING PRIORITIES AND FEASIBLE ASSIGNMENTS

- A main aspect of scheduling in asynchronous RT system is about ***assigning priorities to tasks so that the every task is executed on time, without violating its deadline***
- An assignment of priorities that respect this constraint is called **feasible assignment**
- Example
 - 2 tasks: P1 and P2
 - P1: $p1 = D1 = 2$, $e1 = 1$ ($p1$ = periodo, $D1$ = deadline)
 - must be executed every 2 time units and it needs 1 time unit to execute
 - P2: $p2 = D2 = 5$, $e2 = 2$
 - must be executed every 5 time units and it needs 2 time unit to execute
 - Two possible priority assignments
 - either P1 has a higher priority than P2, or viceversa
 - However: only 1 is feasible
 - $P1 > P2 \Rightarrow$ feasible assignment
 - $P2 > P1 \Rightarrow$ assignment not feasible

TWO MAIN SCHEDULING ALGORITHMS: RM AND EDF

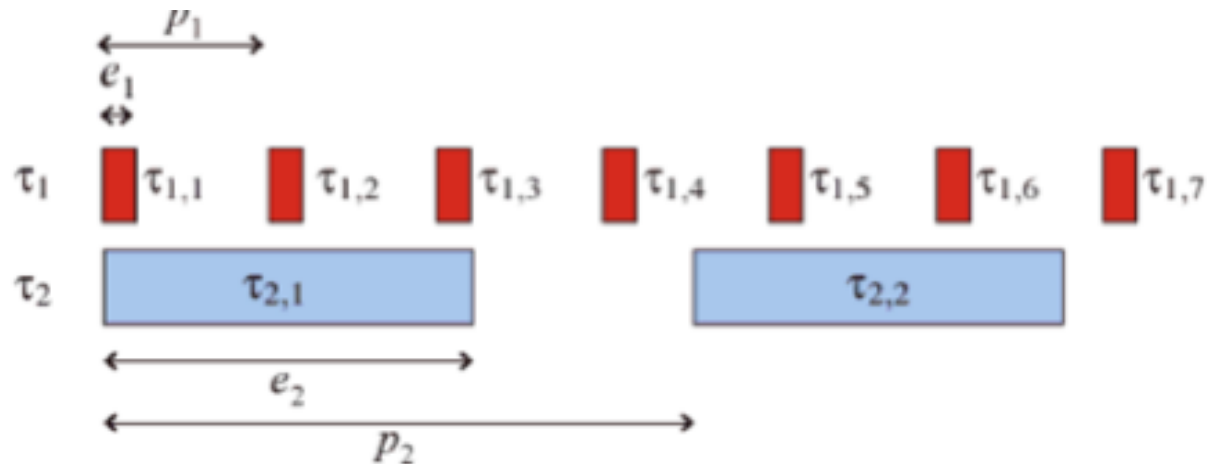
- Different algorithms have been proposed in literature specialised for real-time systems, identifying the necessary and sufficient conditions for feasible assignments
- Two main algorithms
 - **Rate Monotonic (RM)**
 - **Earliest Deadline First (EDF)**

RATE-MONOTONIC (RM)

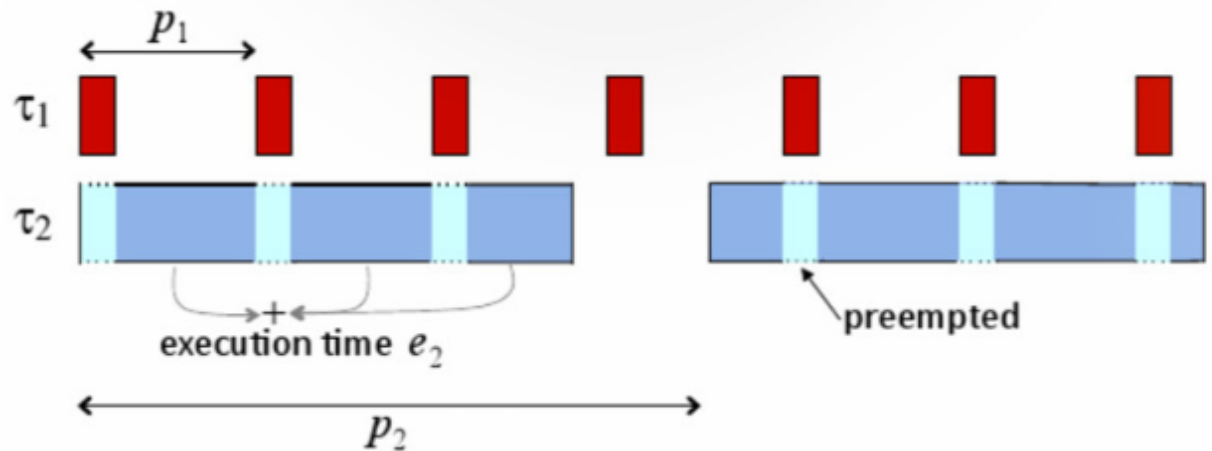
- Scheduling algorithm based on **fixed-priority**
 - priorities defined statically, don't change, and their value is given by the *inverse of the period*
 - so: the smaller is the period, the higher is its priority (in spite of its execution time)
 - for instance:
 - if $p_1 < p_2$, then P_1 is assigned with higher priority
- **Theorem**
 - *given a pre-emptive fixed-priority scheduler and a finite set of periodic tasks $T = \{T_1, T_2, T_3 \dots\}$ with associated periods $P = \{p_1, p_2, p_3, \dots\}$ without precedence constraints, if it exists a feasible assignment, then the assignment defined by the RM is feasible*

EXAMPLES

Two periodic tasks



Pre-emptive scheduling with priorities assigned according to RM



(from [IES], p. 295)

EARLIEST DEADLINE FIRST (EDF)

- This algorithm dynamically computes and changes tasks priorities depending on their deadlines
 - can be used only in dynamic/asynchronous scheduler
- **EDF assigns higher priorities to tasks that have nearer deadlines**
 - Like RM, also for EDF there is an analogous theorem — if it exists a feasible assignment, the EDF finds it under the condition that given a set T of N tasks with CPU usage parameter U is ≤ 1
 - $U = \sum e_i/p_i$, for each task i in $[0..N-1]$
 - e_i = execution time of task i-th, p_i = period of task i-th
- EDF is not always applicable
 - HW tasks, like for interrupt handlers, can require to have fixed priorities, not modifiable
 - EDF implies an overhead on the scheduler that could not be acceptable, in some cases
 - the overhead is given by the fact that task priorities must be recomputed at each scheduling event